ШПАРГАЛКА ДЛЯ АНАЛИТИКА ДАННЫХ

PYTHON
Excel · CSV · Parquet

Подключение, очистка, визуализация и сохранение

Восемь разделов с пошаговыми примерами, кодом, проверками результата и блоками «Совет · Внимание · Примечание» для каждой темы.

ФОРМАТЫ И ФАЙЛЫ



XLSX
openpyxl



CSV
utf-8 · cp1251



PARQUET
pyarrow

ИНСТРУМЕНТЫ И СРЕДЫ



PYTHON
pandas · numpy



JUPYTER
Notebook · Lab



COLAB
Google Drive

АУДИТОРИЯ
Аналитик данных с опытом Python

УРОВЕНЬ
Средний · без базового синтаксиса

СТРУКТУРА
8 разделов · итоговый чек-лист

ОБЪЁМ
50+ страниц · 13 примеров кода

Содержание

1. Введение и подготовка среды **4**

1.1. Используемые библиотеки и их назначение . . . 4

1.2. Команды установки библиотек . . . 5

1.3. Проверка версий и диагностика импорта . . . 5

1.4. Воспроизводимый демонстрационный набор данных . . . 6

2. Подключение к данным **8**

2.1. Чтение из локального каталога . . . 8

2.2. Чтение Excel: листы, диапазоны и движки . . . 9

2.3. Чтение CSV: разделитель, кодировка, десятичный знак . . . 10

2.4. Чтение Parquet: движки, типы и сжатие . . . 11

2.5. Чтение с Google Drive в Colab . . . 12

2.6. Чтение по прямой ссылке . . . 12

2.7. Чтение из публичного GitHub-репозитория . . . 13

3. Первичный осмотр и полезные команды **14**

3.1. Размер, форма и базовая информация . . . 14

3.2. Первые и последние строки, случайные элементы . . . 14

3.3. Типы столбцов и их проверка . . . 15

3.4. Описательная статистика . . . 16

3.5. Пропуски и их распределение . . . 16

3.6. Уникальные значения и категориальные признаки . . . 17

3.7. Выбор строк и столбцов . . . 18

3.8. Сортировка . . . 18

3.9. Полный сценарий первичного осмотра . . . 19

4. Анализ и преобразование данных **20**

4.1. Поиск и обработка пропусков . . . 20

4.2. Поиск и удаление дубликатов . . . 21

4.3. Проверка и преобразование типов . . . 21

4.4. Обработка дат . . . 22

4.5. Очистка текстовых значений . . . 23
4.6. Фильтрация и сортировка с проверкой . . . 24
4.7. Поиск выбросов . . . 25
4.8. Объединение наборов данных . . . 26
4.9. Агрегации и groupby . . . 27
4.10. Сводные таблицы . . . 27
4.11. transform: агрегация с сохранением формы . . . 29
4.12. value_counts как базовый аналитический инструмент . . . 29

5. Целостность и безопасность данных 30

5.1. Дисциплина df_raw, df_work, df_clean . . . 30
5.2. Риски незаметного изменения . . . 31
5.3. Контрольные итоги до и после преобразования . . . 32
5.4. Резервные копии и промежуточные срезы . . . 33
5.5. Сохранение исходных данных в неизменном виде . . . 33
5.6. Проверка размеров, типов, пропусков — три уровня контроля . . . 34
5.7. Защита от потери типов и порядка . . . 35
5.8. Контрольный пример: полный сценарий очистки . . . 36

6. Визуализация 36

6.1. Подготовка данных и общий шаблон . . . 37
6.2. Столбчатая диаграмма . . . 38
6.3. Гистограмма . . . 39
6.4. Boxplot . . . 40
6.5. Линейный график . . . 42
6.6. Тепловая карта корреляций . . . 44
6.7. Сохранение графика в файл . . . 46

7. Сохранение результатов 46

7.1. Сохранение в Excel . . . 46
7.2. Сохранение в CSV . . . 47
7.3. Сохранение в Parquet . . . 48
7.4. Сохранение в локальную папку, папку проекта и Google Drive . . . 48
7.5. Проверка сохранённого файла . . . 49
7.6. Какой формат выбрать . . . 50

8. Работа в Jupyter, Colab, PowerShell и Linux **50**

8.1. Jupyter Notebook 50

8.2. Google Colab 51

8.3. PowerShell (Windows) 52

8.4. Linux и macOS 53

8.5. Клонирование репозитория и первый запуск 53

8.6. Запуск Jupyter в Colab с GitHub 54

9. Итоговый чек-лист **55**

9.1. Чек-лист для PDF-шпаргалки 55

9.2. Чек-лист для Jupyter Notebook 55

9.3. Чек-лист для репозитория на GitHub 56

9.4. Финальные советы 56

1. Введение и подготовка среды

Эта шпаргалка собрана для аналитика данных, который уже уверенно работает с Python и pandas, но хочет систематизировать практические приёмы работы с тремя самыми распространёнными табличными форматами — Excel, CSV и Parquet. Здесь нет базового объяснения синтаксиса Python и устройства DataFrame: мы сосредоточены на рабочих сценариях, важных параметрах функций, типичных ошибках и способах контроля результата. Все примеры построены так, чтобы их можно было запустить без внешних файлов: данные генерируются прямо в коде через `pd.DataFrame({...})`. После этого те же приёмы легко переносятся на реальные наборы, загруженные из вашего источника.

Мы используем три переменные, которые проходят сквозь весь материал: `df_raw` — исходный набор, который мы не меняем; `df_work` — рабочая копия, в которой выполняются очистка и преобразования; `df_clean` — финальный чистый набор, готовый к визуализации и экспорту. Такая дисциплина позволяет сохранить исходные данные в неприкосновенности и в любой момент вернуться к ним, если что-то пошло не так. Дополнительно мы используем `sales_df` как демонстрационный набор для примеров: небольшой синтетический датасет о продажах, который генерируется в одной ячейке.

1.1. Используемые библиотеки и их назначение

В работе с табличными данными в Python есть несколько ключевых библиотек, каждая из которых решает свою задачу. Понимание того, какая библиотека отвечает за какой формат, помогает избежать странных ошибок импорта и неожиданных предупреждений при чтении или записи файлов. Ниже перечислены основные библиотеки и их роли в нашем материале.

- `pandas` — основная библиотека для работы с DataFrame, обеспечивает чтение, запись и преобразование табличных данных.
- `numpy` — числовая база под pandas, используется для расчётов, генерации данных и работы с массивами.
- `openpyxl` — движок для чтения и записи современных `.xlsx` файлов; нужен для большинства операций с Excel.
- `pyarrow` — основной движок для работы с Parquet, обеспечивает быстрое чтение и запись с поддержкой типизации.
- `fastparquet` — альтернативный движок для Parquet, иногда удобен, если `pyarrow` недоступен.
- `matplotlib` — базовая библиотека визуализации, даёт полный контроль над параметрами графика.
- `seaborn` — надстройка над matplotlib, упрощает статистическую визуализацию.

> **СОВЕТ. Не ставьте все библиотеки "на всякий случай"**
>
> Если вы работаете только с CSV, вам не нужны openpyxl и pyarrow. Лишние зависимости утяжеляют виртуальное окружение и усложняют воспроизведение. Ставьте только то, что реально используется в проекте.

Для каждого формата файлов важно понимать, какой движок используется под капотом. Для Excel это обычно `openpyxl` (формат `.xlsx`), а для Parquet — `pyarrow` (рекомендуется) или `fastparquet`. CSV читается встроенным парсером pandas и не требует внешних движков. Если нужный движок не установлен, pandas выдаст понятное сообщение об ошибке с указанием имени библиотеки, которую нужно установить.

1.2. Команды установки библиотек

Установка библиотек немного отличается в разных средах. Ниже приведены команды для основных сред, с которыми работает аналитик: Jupyter Notebook, Google Colab, PowerShell (Windows) и терминал Linux/macOS. Во всех случаях используется `pip` — стандартный пакетный менеджер Python. Если вы используете `conda`, команды можно адаптировать, но мы ориентируемся на минимальный совместимый набор.

Jupyter Notebook

```
# В ячейке Jupyter Notebook (или JupyterLab) используем магию !
!pip install pandas numpy openpyxl pyarrow matplotlib seaborn

# Обновление до актуальных версий
!pip install --upgrade pandas openpyxl pyarrow
```

Google Colab

```
# В ячейке Colab также используем ! перед командой pip
!pip install pandas numpy openpyxl pyarrow matplotlib seaborn

# Colab обновляется редко, но если нужно:
!pip install --upgrade pandas
```

PowerShell

```
# В PowerShell (Windows) — без !, обычная команда pip
pip install pandas numpy openpyxl pyarrow matplotlib seaborn

# Обновление
pip install --upgrade pandas openpyxl pyarrow
```

Linux

```
# В терминале Linux или macOS
pip install pandas numpy openpyxl pyarrow matplotlib seaborn

# Часто удобно ставить в пользователя, если не активировано venv
pip install --user pandas openpyxl pyarrow

# Обновление
pip install --upgrade pandas openpyxl pyarrow
```

> **ВНИМАНИЕ. Не смешивайте системный Python и виртуальное окружение**
>
> Установка пакетов напрямую в системный Python (особенно на Linux) может привести к конфликтам с системными зависимостями. В Debian/Ubuntu и других дистрибутивах это иногда вызывает ошибку `externally-managed-environment`. Используйте виртуальное окружение (см. раздел 8) или флаг `--break-system-packages` только как временную меру.

1.3. Проверка версий и диагностика импорта

Перед началом работы полезно убедиться, что все библиотеки установлены и импортируются без ошибок. Это особенно важно при первом запуске ноутбука на новой машине или в Colab. Простая проверка версий занимает одну ячейку и сразу выявляет проблемы: несовместимость версий, отсутствие движка, неверно настроенное окружение. Несоответствие версий часто становится причиной непонятных ошибок, когда код работает у одного аналитика и не работает у другого.

Jupyter Notebook

```
import pandas as pd
import numpy as np
import openpyxl
import pyarrow
import matplotlib
import seaborn as sns

print("pandas:", pd.__version__)
print("numpy:", np.__version__)
print("openpyxl:", openpyxl.__version__)
print("pyarrow:", pyarrow.__version__)
print("matplotlib:", matplotlib.__version__)
print("seaborn:", sns.__version__)
```

> **ПРИМЕЧАНИЕ. Что должно получиться**
>
> Версии могут отличаться, но для актуальных на момент подготовки материала достаточно ориентироваться на pandas ≥ 2.0, openpyxl ≥ 3.1, pyarrow ≥ 14.0, matplotlib ≥ 3.7, seaborn ≥ 0.13. Если какая-то версия намного меньше, обновите пакет командой `pip install --upgrade <имя>`.

Для диагностики частых ошибок импорта полезно выполнить короткую последовательность проверок. Если `import pandas` падает с `ModuleNotFoundError`, значит библиотека не установлена в текущем окружении. Если `pd.read_excel()` выдаёт `ImportError: Missing optional dependency 'openpyxl'`, нужно установить движок Excel. Аналогично для Parquet: ошибка `Missing optional dependency 'pyarrow'` лечится командой `pip install pyarrow`. Важно проверить, что вы используете именно то окружение, в котором устанавливали пакеты: в Jupyter и Colab можно посмотреть путь к Python через `sys.executable`, а в терминале — `which python` или `where python` (Windows).

1.4. Воспроизводимый демонстрационный набор данных

Все примеры в этом материале построены на одном синтетическом наборе `sales_df`, который генерируется прямо в коде. Это позволяет запускать примеры без внешних файлов и сразу видеть результат. Структура набора имитирует типичный лог продаж: идентификатор заказа, дата, клиент, категория товара, сумма, количество, регион и канал продаж. В данных намеренно есть пропуски, дубликаты и спорные значения — они нужны для демонстрации приёмов очистки.

Jupyter Notebook

```
import pandas as pd
import numpy as np

# Воспроизводимость: фиксируем генератор случайных чисел
rng = np.random.default_rng(seed=42)

n = 30
sales_df = pd.DataFrame({
"order_id": [f"ORD-{i+1:04d}" for i in range(n)],
"order_date": pd.date_range(start="2024-01-01", periods=n, freq="D"),
"client": rng.choice(["Alice", "Bob", "Charlie", "Diana", np.nan], size=n),
"category": rng.choice(["Books", "Electronics", "Clothing", "Books"], size=n),
"amount": np.round(rng.normal(loc=120, scale=40, size=n), 2),
"quantity": rng.integers(low=1, high=6, size=n),
"region": rng.choice(["Moscow", "Saint Petersburg", "Kazan", "Novosibirsk"], size=n),
"channel": rng.choice(["Online", "Retail", "Online"], size=n),
})

# Намеренно вводим несколько пропусков и один дубликат для примеров очистки
sales_df.loc[3, "amount"] = np.nan
sales_df.loc[7, "client"] = np.nan
sales_df.loc[10, "quantity"] = np.nan
sales_df = pd.concat([sales_df, sales_df.iloc[[0]]], ignore_index=True)

print(sales_df.shape)
sales_df.head()
```

СОВЕТ. Что делает каждый параметр

`np.random.default_rng(seed=42)` создаёт генератор с фиксированным зерном — результаты повторяются между запусками. `pd.date_range(..., freq="D")` создаёт диапазон дней с шагом 1. `rng.choice(...)` выбирает значения из списка с заданными вероятностями. `rng.normal(loc=120, scale=40)` генерирует нормальное распределение со средним 120 и стандартным отклонением 40. `np.round(..., 2)` округляет до двух знаков — удобно для денежных сумм.



Один и тот же seed даёт одинаковые данные на любой машине и в любом запуске

Получившийся `sales_df` мы будем использовать на протяжении всех примеров. Когда будем показывать приёмы очистки, этот же набор будет служить исходным. Если в вашем реальном проекте структура отличается, вы просто подставляете свой DataFrame — все операции универсальны и не зависят от конкретных имён столбцов. На практике аналитик часто начинает с небольшого прототипа на синтетических данных, отлаживает логику, а затем применяет тот же код к настоящему набору.

2. Подключение к данным

Подключение к данным — это первый шаг любого аналитического сценария. Источников много: локальный файл, сетевой диск, Google Drive в Colab, прямая ссылка на файл в интернете, репозиторий GitHub. Мы разберём каждый формат (Excel, CSV, Parquet) и каждый распространённый источник, с пояснением параметров и проверкой результата. Главная цель этого раздела — научиться уверенно читать данные из любого источника и понимать, какие параметры отвечают за корректность разбора.

2.1. Чтение из локального каталога

Самый частый сценарий — файл лежит в папке проекта или в подкаталоге `data/`. Для всех трёх форматов pandas предоставляет отдельные функции: `read_excel`, `read_csv`, `read_parquet`. Важно явно указывать путь и проверять, что pandas действительно открыл файл, а не создал пустой DataFrame из-за неверного аргумента. Перед чтением полезно убедиться, что файл существует — это экономит время на диагностике.

Jupyter Notebook

```
import pandas as pd
from pathlib import Path

# Путь к файлу относительно текущего каталога ноутбука
excel_path = Path("data/sales.xlsx")
csv_path = Path("data/sales.csv")
parquet_path = Path("data/sales.parquet")

# Проверка существования
print("Excel exists:", excel_path.exists())
print("CSV exists:", csv_path.exists())
print("Parquet exists:", parquet_path.exists())

# Чтение
df_from_excel = pd.read_excel(excel_path)
df_from_csv = pd.read_csv(csv_path)
df_from_parquet = pd.read_parquet(parquet_path)

# Проверка размерности
print(df_from_excel.shape, df_from_csv.shape, df_from_parquet.shape)
```

СОВЕТ. Используйте pathlib, а не строки

Класс `pathlib.Path` корректно работает с путями на Windows, Linux и macOS, поддерживает `/` для конкатенации и имеет удобные методы `exists()`, `parent`, `name`, `suffix`. Строковые пути с `os.path.join` тоже работают, но менее читаемы и часто приводят к ошибкам на кросс-платформенном коде.

2.2. Чтение Excel: листы, диапазоны и движки

Excel-файл может содержать несколько листов, и `pd.read_excel()` по умолчанию читает первый. Если нужный лист не первый, его имя или индекс передаются через параметр `sheet_name`. Дополнительно доступны параметры для указания диапазона ячеек (`usecols`, `skiprows`, `nrows`) и выбора заголовка (`header`). Все эти параметры комбинируются и позволяют точно настроить, какую часть листа читать.



Jupyter Notebook

```
# Чтение конкретного листа по имени
df_sheet1 = pd.read_excel("data/sales.xlsx", sheet_name="Sales 2024")

# Чтение листа по индексу (0 — первый лист)
df_sheet2 = pd.read_excel("data/sales.xlsx", sheet_name=1)

# Чтение диапазона столбцов: A:D, пропуск первых двух строк
df_range = pd.read_excel(
"data/sales.xlsx",
sheet_name="Sales 2024",
usecols="A:D",
skiprows=2,
)

# Заголовок не в первой строке
df_no_header = pd.read_excel(
"data/sales.xlsx",
sheet_name="Sales 2024",
header=3,
)

# Получить словарь всех листов сразу
all_sheets = pd.read_excel("data/sales.xlsx", sheet_name=None)
print("Листы:", list(all_sheets.keys()))
```

ПРИМЕЧАНИЕ. Какой движок используется

По умолчанию для `.xlsx` pandas использует `openpyxl`. Для старых `.xls` файлов применяется `xlrd` (только чтение). Для записи `.xlsx` тоже используется `openpyxl`. Если нужно явно указать движок: `pd.read_excel(..., engine="openpyxl")`. Это полезно, когда в проекте установлено несколько движков и вы хотите избежать неявного выбора.

ВНИМАНИЕ. Осторожно с параметрами header и skiprows вместе

Если вы используете `skiprows=2` и `header=0`, фактически заголовок берётся из третьей строки файла (индекс 2). Частая ошибка — перепутать `skiprows` и `header`: первый пропускает строки до чтения, второй указывает, какая строка станет заголовком. Проверяйте результат через `df.columns` сразу после чтения.

2.3. Чтение CSV: разделитель, кодировка, десятичный знак

CSV кажется простым форматом, но на практике вызывает больше всего проблем. Разделитель может быть запятой, точкой с запятой, табуляцией; кодировка — UTF-8, Windows-1251, Latin-1; десятичный разделитель — точка или запятая. Если данные выгружены из русской версии Excel, разделитель почти всегда `;`, а десятичная запятая. Если файл из англоязычного источника — наоборот. Понимание этих параметров критично для корректного чтения.



Разделитель (sep), кодировка (encoding) и десятичный знак (decimal) — три критичных параметра

Jupyter Notebook

```
# Стандартный CSV с запятой, UTF-8
df_csv = pd.read_csv("data/sales.csv")

# Русский CSV: разделитель ;, кодировка Windows-1251, запятая как десятичный знак
df_csv_ru = pd.read_csv(
"data/sales_ru.csv",
sep=";",
encoding="cp1251",
decimal=",",
thousands=".",
)

# CSV с табуляцией (часто называется .tsv)
df_tsv = pd.read_csv("data/sales.tsv", sep="\t")

# CSV без заголовка — имена столбцов задаются вручную
df_no_header = pd.read_csv(
"data/sales_noheader.csv",
header=None,
names=["order_id", "date", "amount", "client"],
)
```

СОВЕТ. Если кодировка неизвестна

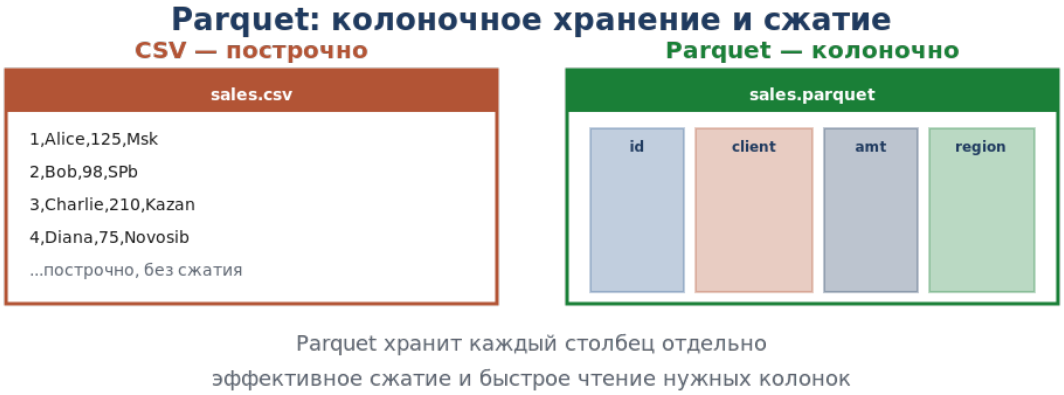
Начните с `encoding="utf-8"`. Если падает `UnicodeDecodeError`, попробуйте `"cp1251"` (Windows для русского), затем `"latin-1"` (читает что угодно, но кириллица будет искажена). Для серьёзной диагностики используйте библиотеку `chardet`: `import chardet; chardet.detect(open('file.csv','rb').read(10000))`.

ВНИМАНИЕ. Десятичный знак и локаль

Если в CSV числа записаны как `1 234,56` (русский формат), обязательно укажите `decimal=","` и `thousands="."`. Без этого столбец прочитается как строки или как числа с потерей точности. После чтения проверьте `df.dtypes` — числовой столбец должен быть `float64`, а не `object`.

2.4. Чтение Parquet: движки, типы и сжатие

Parquet — это колоночный формат хранения с поддержкой типизации и эффективного сжатия. Он особенно полезен для больших наборов: файлы в 5-10 раз меньше CSV и читаются заметно быстрее. Pandas использует для чтения Parquet движок `pyarrow` (по умолчанию) или `fastparquet`. Движок можно выбрать через параметр `engine`. Parquet сохраняет типы столбцов, поэтому после чтения вы получаете DataFrame с уже корректными типами, без необходимости в `astype()`.



Jupyter Notebook

```
# Чтение Parquet (движок по умолчанию — pyarrow)
df_pq = pd.read_parquet("data/sales.parquet")

# Явное указание движка
df_pq_fp = pd.read_parquet("data/sales.parquet", engine="fastparquet")

# Чтение только нужных столбцов — ускоряет работу с большими файлами
df_pq_cols = pd.read_parquet("data/sales.parquet", columns=["order_id", "amount"])

# Чтение с фильтром на уровне файла (pushdown)
df_pq_filtered = pd.read_parquet(
"data/sales.parquet",
filters=[("region", "==", "Moscow")],
)
```

СОВЕТ. columns и filters — главные друзья на больших данных

Параметр `columns` загружает только указанные столбцы, экономя память. Параметр `filters` применяет условие ещё на этапе чтения, до загрузки в DataFrame. Для файла на 10 миллионов строк и 50 столбцов это разница между секундами и минутами.

ПРИМЕЧАНИЕ. Какой движок выбрать

`pyarrow` — рекомендуется по умолчанию: быстрее, активнее развивается, лучше поддерживает типы. `fastparquet` полезен, если у вас старые файлы, записанные через `fastparquet`, или есть специфические требования. В большинстве случаев достаточно `pyarrow`.

2.5. Чтение с Google Drive в Colab

В Google Colab для работы с файлами Google Drive используется модуль `google.colab.drive`. После монтирования диск доступен по пути `/content/drive/MyDrive/`. Это удобный способ хранить рабочие файлы между сессиями Colab и использовать их в нескольких ноутбуках. После монтирования все пути работают как обычно через `pd.read_*`.

Google Colab

```
from google.colab import drive
drive.mount("/content/drive")

# После авторизации файлы вашего Drive доступны по пути:
df_drive = pd.read_excel("/content/drive/MyDrive/projects/sales.xlsx")

# Чтение CSV с Drive
df_drive_csv = pd.read_csv("/content/drive/MyDrive/projects/sales.csv")

# Чтение Parquet
df_drive_pq = pd.read_parquet("/content/drive/MyDrive/projects/sales.parquet")

# Проверка
print(df_drive.shape)
df_drive.head()
```

ВНИМАНИЕ. Drive требует авторизации

При вызове `drive.mount(...)` Colab откроет окно авторизации. После подтверждения монтирование происходит автоматически. Если вы не авторизованы — будет ошибка `TransportError` или `RefreshError`. Проверьте, что вы вошли в тот же аккаунт Google, которому принадлежит файл.

2.6. Чтение по прямой ссылке

Если файл доступен по HTTP/HTTPS, pandas может прочитать его напрямую без скачивания. Это работает для CSV и Parquet, а для Excel — через дополнительный параметр или через скачивание во временный файл. Подход удобен для открытых данных, регулярных обновлений и интеграции с публичными источниками. Важно учитывать, что большие файлы по медленной сети читаются долго, и лучше кешировать их локально.

Jupyter Notebook

```
# CSV по прямой ссылке
url_csv = "https://raw.githubusercontent.com/USERNAME/REPOSITORY/main/data/sales.csv"
df_url = pd.read_csv(url_csv)

# Parquet по ссылке
url_pq = "https://raw.githubusercontent.com/USERNAME/REPOSITORY/main/data/sales.parquet"
df_pq_url = pd.read_parquet(url_pq)

# Excel по ссылке — сохраняем во временный файл
import urllib.request, tempfile
url_xlsx = "https://github.com/USERNAME/REPOSITORY/raw/main/data/sales.xlsx"
tmp_path = tempfile.NamedTemporaryFile(suffix=".xlsx").name
urllib.request.urlretrieve(url_xlsx, tmp_path)
df_xlsx_url = pd.read_excel(tmp_path)
```

СОВЕТ. raw.githubusercontent.com vs github.com

Для чтения через pandas используйте ссылки `raw.githubusercontent.com/...` или `github.com/.../raw/...`. Обычная ссылка `github.com/.../blob/...` отдаёт HTML-страницу с превью файла, а не сам файл. Это типичная ошибка, из-за которой чтение падает с ошибкой парсинга.

2.7. Чтение из публичного GitHub-репозитория

GitHub удобен для хранения небольших наборов данных в открытом доступе. Файл можно читать по прямой ссылке (см. предыдущий раздел) или клонировать репозиторий целиком. Второй способ удобен, когда в проекте несколько связанных файлов или нужно запустить ноутбук, который ссылается на локальные пути. Чтение по ссылке — для разовых операций; клонирование — для воспроизводимого проекта.

PowerShell

```
# Клонирование репозитория в PowerShell
git clone https://github.com/USERNAME/REPOSITORY.git
cd REPOSITORY
# Дальше в Jupyter или Python можно читать локальные пути:
# pd.read_csv("data/sales.csv")
```

Linux

```
# Клонирование в терминале Linux/macOS
git clone https://github.com/USERNAME/REPOSITORY.git
cd REPOSITORY
# pd.read_csv("data/sales.csv")
```

Jupyter Notebook

```
# Вариант: чтение отдельных файлов по raw-ссылке без клонирования
import pandas as pd
base = "https://raw.githubusercontent.com/USERNAME/REPOSITORY/main/data"
df_repo_csv = pd.read_csv(f"{base}/sales.csv")
df_repo_pq = pd.read_parquet(f"{base}/sales.parquet")
print(df_repo_csv.shape, df_repo_pq.shape)
```

ПРИМЕЧАНИЕ. Замените USERNAME и REPOSITORY на свои значения

Во всех примерах ссылок используются шаблоны `USERNAME` и `REPOSITORY`. В реальном проекте подставьте свой логин GitHub и имя репозитория. Не пытайтесь использовать эти шаблоны как есть — они приведут к 404.

3. Первичный осмотр и полезные команды

После успешного чтения данных следующий шаг — первичный осмотр. Это быстрый, но обязательный ритуал: проверить размер, типы колонок, наличие пропусков, посмотреть первые и последние строки, оценить распределение числовых признаков. Пропуск этого шага часто приводит к сюрпризам: неожиданным строковым значениям в числовых столбцах, дубликатам, неверным типам дат. В этом разделе собраны команды, которые стоит запускать сразу после чтения любого нового файла.

3.1. Размер, форма и базовая информация

Самые первые проверки — размер DataFrame и общая структура. Команда `shape` возвращает кортеж `(rows, cols)`, `len(df)` — число строк, `df.size` — общее число ячеек. Команда `info()` выводит типы столбцов, число непустых значений и использование памяти. Этих трёх вызовов достаточно, чтобы за 10 секунд понять, что перед нами.

Jupyter Notebook

```
# Размер и форма
print("shape:", sales_df.shape)
print("rows:", len(sales_df))
print("cols:", sales_df.columns.tolist())
print("size:", sales_df.size)

# Общая информация о типах и памяти
sales_df.info()

# Использование памяти с учётом реальных типов
print(sales_df.memory_usage(deep=True))
```

СОВЕТ. info(memory_usage="deep")

По умолчанию `info()` показывает приблизительное использование памяти. Если в данных есть строковые столбцы (тип `object`), реальное потребление может быть в 5-10 раз больше. Передайте `memory_usage="deep"` или вызовите `df.memory_usage(deep=True)` для точной оценки. Это особенно полезно при работе с большими наборами.

3.2. Первые и последние строки, случайные элементы

Команды `head()`, `tail()` и `sample()` позволяют быстро увидеть структуру данных. `head()` по умолчанию возвращает первые 5 строк, `tail()` — последние 5. `sample()` выбирает случайные строки, что полезно для проверки, что данные перемешаны и нет скрытой сортировки, влияющей на анализ. Все три команды принимают аргумент `n` для указания числа строк.

Jupyter Notebook

```
# Первые строки
sales_df.head()
sales_df.head(10)

# Последние строки
sales_df.tail()
sales_df.tail(3)

# Случайная выборка — для проверки, что данные перемешаны
sales_df.sample(5, random_state=42)

# Случайная выборка 10% данных
sales_df.sample(frac=0.1, random_state=42)
```

ПРИМЕЧАНИЕ. Зачем sample, если есть head

`head()` показывает первые строки — часто это отсортированные данные или данные за начало периода. `sample()` даёт более репрезентативный срез и помогает заметить аномалии, скрытые в середине набора. Хорошая практика — после чтения делать и `head()`, и `sample()`.

3.3. Типы столбцов и их проверка

Знание типов столбцов — основа корректной работы. Числовые столбцы должны быть `int64` или `float64`, строки — `object` или `string`, даты — `datetime64`. Если числовой столбец оказался типа `object`, скорее всего при чтении возникла проблема с кодировкой, десятичным разделителем или пропущенными значениями. Проверка типов после чтения и после каждого преобразования — обязательный шаг.

Jupyter Notebook

```
# Типы столбцов
print(sales_df.dtypes)

# Сводка по типам: сколько столбцов каждого типа
print(sales_df.dtypes.value_counts())

# Выбор столбцов определённого типа
numeric_cols = sales_df.select_dtypes(include="number").columns
text_cols = sales_df.select_dtypes(include="object").columns
date_cols = sales_df.select_dtypes(include="datetime").columns

print("Числовые:", numeric_cols.tolist())
print("Текстовые:", text_cols.tolist())
print("Даты:", date_cols.tolist())
```

ВНИМАНИЕ. object не всегда означает строку

Тип `object` в pandas — это массив Python-объектов. В нём могут лежать строки, но также смешанные значения (числа и строки, числа и NaN). Если числовой столбец оказался `object`, проверьте `df["col"].apply(type).value_counts()` — это покажет, какие именно типы смешаны в столбце.

3.4. Описательная статистика

Команда `describe()` даёт быструю сводку по числовым столбцам: количество, среднее, стандартное отклонение, минимум, квартили, максимум. С параметром `include="all"` она также показывает информацию по нечисловым столбцам: число уникальных значений, самое частое, частоту. Это самый быстрый способ оценить, есть ли аномалии в данных — выбросы, экстремальные значения, неожиданные распределения.

Jupyter Notebook

```
# Числовая статистика
sales_df.describe()

# Включая нечисловые столбцы
sales_df.describe(include="all")

# Только текстовые столбцы
sales_df.describe(include="object")

# Процентили на свой выбор
sales_df.describe(percentiles=[0.01, 0.1, 0.5, 0.9, 0.99])

# Агрегаты по конкретным столбцам
sales_df[["amount", "quantity"]].describe()
```

СОВЕТ. Обращайте внимание на разницу mean и median

Если среднее сильно отличается от медианы (50%), распределение скошено или есть выбросы. Например, `mean=120, 50%=80, max=2000` — почти наверняка есть несколько экстремальных значений. Это первый сигнал к проверке на выбросы.

3.5. Пропуски и их распределение

Пропуски — постоянный спутник реальных данных. Важно знать, где они, сколько их и есть ли в них закономерность. Команда `isna().sum()` показывает число пропусков по столбцам, `isna().mean()` — долю пропусков. Сортировка по этому значению помогает выделить столбцы, которые почти полностью пустые и могут быть удалены.

Jupyter Notebook

```
# Число пропусков по столбцам
print(sales_df.isna().sum())

# Доля пропусков
print(sales_df.isna().mean().round(3))

# Сортировка по доле пропусков (от больших к меньшим)
print((sales_df.isna().mean().sort_values(ascending=False).round(3) * 100))

# Строки с любым пропуском
mask_na = sales_df.isna().any(axis=1)
print("Строк с хотя бы одним пропуском:", mask_na.sum())

# Полностью пустые строки
print("Полностью пустых строк:", sales_df.isna().all(axis=1).sum())

# Столбцы без единого пропуска
print("Столбцы без пропусков:", sales_df.columns[sales_df.notna().all()].tolist())
```

ПРИМЕЧАНИЕ. axis=1 — по строкам, axis=0 — по столбцам

В pandas параметр `axis=1` означает "по строкам" (применяем функцию к каждой строке), `axis=0` — "по столбцам". `isna().any(axis=1)` возвращает Series, где для каждой строки указано, есть ли в ней хотя бы один пропуск.

3.6. Уникальные значения и категориальные признаки

Для категориальных столбцов полезно знать, какие значения в них встречаются и с какой частотой. Команды `nunique()`, `unique()` и `value_counts()` дают полный обзор. Это помогает заметить опечатки в категориях (например, "Books" и "books "), скрытые значения (`np.nan`, `"nan"` как строка), и понять, нужно ли объединять редкие категории.

Jupyter Notebook

```
# Число уникальных значений по столбцам
print(sales_df.nunique())

# Уникальные значения конкретного столбца
print(sales_df["region"].unique())

# Частоты значений
print(sales_df["category"].value_counts())

# Частоты с долями
print(sales_df["category"].value_counts(normalize=True).round(3))

# Частоты с пропусками
print(sales_df["category"].value_counts(dropna=False))
```

СОВЕТ. Проверяйте "почти дубликаты" в категориях

Если в `value_counts()` вы видите "Books" и "Books " (с пробелом) — это две разные категории. Используйте `df["col"].str.strip()` для очистки пробелов. Аналогично проверяйте регистр: "Books" и "books" — разные значения. Их можно объединить через `str.title()` или `str.upper()`.

3.7. Выбор строк и столбцов

pandas предоставляет несколько способов выбора данных: `loc` по меткам, `iloc` по позициям, прямую индексацию по столбцам. Для аналитика важно понимать разницу и использовать подходящий способ в каждой ситуации. `loc` поддерживает логические условия, что делает его основным инструментом фильтрации. `iloc` удобен для позиционной выборки, когда имена не важны.

Jupyter Notebook

```
# Выбор столбцов по имени
df_cols = sales_df[["order_id", "amount", "region"]]

# loc: по меткам и условиям
df_loc = sales_df.loc[sales_df["amount"] > 100, ["order_id", "client"]]

# iloc: по позициям (первые 5 строк, первые 3 столбца)
df_iloc = sales_df.iloc[:5, :3]

# Несколько условий
df_filtered = sales_df[
(sales_df["amount"] > 100) &
(sales_df["region"] == "Moscow") &
(sales_df["channel"] == "Online")
]

# Условие с isin
df_in = sales_df[sales_df["region"].isin(["Moscow", "Kazan"])]

# Условие с исключением
df_notin = sales_df[~sales_df["region"].isin(["Moscow", "Kazan"])]
```

ВНИМАНИЕ. Осторожно с цепной индексацией

Конструкция `df[df["a"] > 0]["b"] = 0` вызывает `SettingWithCopyWarning` и может не изменить DataFrame. Используйте `loc`: `df.loc[df["a"] > 0, "b"] = 0`. Это работает гарантированно и не вызывает предупреждений.

3.8. Сортировка

Сортировка — простая, но важная операция. `sort_values()` сортирует по одному или нескольким столбцам, `sort_index()` — по индексу. Параметр `ascending` управляет направлением (по умолчанию `True` — по возрастанию). Параметр `na_position` определяет, где будут пропуски — в начале или в конце. Сортировка помогает найти экстремальные значения, проверить хронологию, подготовить данные к визуализации.

Jupyter Notebook

```
# Сортировка по одному столбцу
sales_df.sort_values("amount")

# По убыванию
sales_df.sort_values("amount", ascending=False)

# По нескольким столбцам (по убыванию amount, по возрастанию date)
sales_df.sort_values(["amount", "order_date"], ascending=[False, True])

# Сортировка по индексу
sales_df.sort_index()

# Пропуски в начале
sales_df.sort_values("amount", na_position="first")

# Сортировка по строке (по значениям в одной строке)
sales_df.loc[:, "amount":"quantity"].sort_values(by="amount", axis=0)
```

СОВЕТ. Не изменяйте исходный DataFrame при сортировке

Все операции `sort_values` по умолчанию возвращают новый DataFrame. Параметр `inplace=True` изменяет исходный, но в нашей дисциплине `df_raw` мы не трогаем. Для рабочей копии `df_work` можно использовать `inplace=True`, но чаще удобнее явно переназначать: `df_work = df_work.sort_values("amount")`.

3.9. Полный сценарий первичного осмотра

Соберём все команды в один блок, который стоит запускать сразу после чтения нового файла. Это шаблон быстрой проверки, который экономит время и помогает заметить аномалии до того, как они станут проблемой. Сохраните его как отдельную ячейку в начале каждого аналитического ноутбука.

Jupyter Notebook

```
def quick_inspect(df: pd.DataFrame, name: str = "df") -> None:
'''Быстрый осмотр DataFrame после чтения.'''
print(f"=== {name} ===")
print(f"shape: {df.shape}")
print(f"\ndtypes:\n{df.dtypes}")
print(f"\nПервые строки:\n{df.head()}")
print(f"\nОписательная статистика:\n{df.describe(include='all')}")
print(f"\nПропуски по столбцам:\n{df.isna().sum()}")
print(f"\nУникальные значения:\n{df.nunique()}")

# Использование
quick_inspect(sales_df, "sales_df")
```

ПРИМЕЧАНИЕ. Шаблон для всех проектов

Функцию `quick_inspect` можно скопировать в любой проект. Она запускается за секунду и даёт полный снимок данных. Альтернатива — вынести её в отдельный модуль `utils.py` в репозитории и импортировать: `from utils import quick_inspect`.

4. Анализ и преобразование данных

После первичного осмотра начинается этап активной работы с данными: обработка пропусков, дубликатов, типов, дат, текста, выбросов, объединений и агрегаций. Каждый шаг должен сопровождаться проверкой результата — иначе легко незаметно потерять строки, изменить тип или сломать индекс. В этом разделе мы проходим все ключевые приёмы и для каждого показываем, как убедиться, что преобразование сработало как ожидалось.

4.1. Поиск и обработка пропусков

Пропуски в реальных данных — норма, а не исключение. Важно понимать, что с ними делать: удалить, заполнить константой, заполнить статистикой (медиана, среднее, мода) или оставить как есть, если для анализа они не критичны. Выбор стратегии зависит от доли пропусков, природы столбца и цели анализа. Главное правило — любое действие с пропусками должно сопровождаться проверкой `isna().sum()` до и после.

Jupyter Notebook

```
# Рабочая копия — исходный sales_df не трогаем
df_work = sales_df.copy()

# Сколько пропусков
print(df_work.isna().sum())

# Удаление строк с любым пропуском
df_drop_any = df_work.dropna()
print("Было:", len(df_work), "Стало:", len(df_drop_any))

# Удаление строк, где пропуск в конкретном столбце
df_drop_amount = df_work.dropna(subset=["amount"])

# Удаление столбцов, где все значения — пропуски
df_drop_cols = df_work.dropna(axis=1, how="all")

# Заполнение константой
df_fill_const = df_work.fillna({"client": "Unknown", "amount": 0})

# Заполнение статистикой
median_amount = df_work["amount"].median()
df_fill_med = df_work.copy()
df_fill_med["amount"] = df_work["amount"].fillna(median_amount)

# Заполнение вперёд (forward fill) — для временных рядов
df_ffill = df_work.sort_values("order_date").ffill()

# Заполнение назад (backward fill)
df_bfill = df_work.sort_values("order_date").bfill()
```

ВНИМАНИЕ. fillna без словаря меняет все столбцы

`df.fillna(0)` заполнит нулями все пропуски во всех столбцах. В числовых столбцах это нормально, но в строковых появится "0" как строка. Всегда передавайте словарь: `df.fillna({"col_num": 0, "col_str": "Unknown"})`.

СОВЕТ. Проверка после заполнения

После любого заполнения убедитесь, что результат именно тот, что вы задумали: `df["col"].isna().sum()` должно вернуть 0, а `df["col"].value_counts()` — показать новое значение. Частая ошибка — заполнить "мало", а потом обнаружить, что остались редкие NaN в неожиданных местах.

4.2. Поиск и удаление дубликатов

Дубликаты бывают двух типов: полные (вся строка повторяется) и по подмножеству столбцов (например, два заказа с одинаковым `order_id`). `duplicated()` возвращает логическую маску, `drop_duplicates()` сразу удаляет. Параметр `keep` управляет тем, какую копию оставить: `"first"` (по умолчанию), `"last"` или `False` (удалить все дубликаты).

Jupyter Notebook

```
# Число дубликатов (полных)
print("Полных дубликатов:", df_work.duplicated().sum())

# Дубликаты по подмножеству столбцов
print("Дубли по order_id:", df_work.duplicated(subset=["order_id"]).sum())

# Удаление полных дубликатов
df_dedup = df_work.drop_duplicates()

# Удаление дубликатов по конкретным столбцам, оставить последний
df_dedup_id = df_work.drop_duplicates(subset=["order_id"], keep="last")

# Удаление всех дубликатов (оставить только уникальные)
df_only_unique = df_work.drop_duplicates(subset=["order_id"], keep=False)

# Проверка
print("После удаления:", len(df_dedup), "Было:", len(df_work))
print("Осталось дубликатов:", df_dedup.duplicated().sum())
```

ПРИМЕЧАНИЕ. keep=False для аналитики

Если вы хотите найти "подозрительные" строки, у которых есть двойник — используйте `keep=False`. Это вернёт все строки, у которых есть хотя бы один дубликат, без выбора "основного". Удобно для аудита данных.

4.3. Проверка и преобразование типов

Корректные типы столбцов — основа корректной аналитики. Числа в строковом виде не агрегируются, даты в строковом виде не поддерживают временные операции, категории в строковом виде занимают больше памяти. После чтения всегда проверяйте типы и при необходимости преобразуйте через `astype()`, `to_datetime()`, `to_numeric()`.

Jupyter Notebook

```
# Текущие типы
print(df_work.dtypes)

# Преобразование числового столбца, который оказался строкой
df_work["amount_num"] = pd.to_numeric(df_work["amount"], errors="coerce")

# Преобразование строки в дату
df_work["order_date_dt"] = pd.to_datetime(df_work["order_date"], errors="coerce")

# Преобразование строки в категорию (экономит память)
df_work["category_cat"] = df_work["category"].astype("category")

# Преобразование в int с обработкой пропусков (int не поддерживает NaN)
df_work["quantity_int"] = df_work["quantity"].fillna(0).astype(int)

# Проверка
print(df_work[["amount", "amount_num", "order_date", "order_date_dt"]].dtypes)
```

ВНИМАНИЕ. int и NaN

Стандартный `int64` в pandas не поддерживает пропуски. Если в столбце есть NaN, `astype(int)` упадёт. Используйте `Int64` (с большой буквы) — это Nullable Integer type, который допускает пропуски: `df["col"].astype("Int64")`.

СОВЕТ. errors="coerce" — удобное падение

Параметр `errors="coerce"` в `to_numeric` и `to_datetime` превращает не разбираемые значения в `NaN`, а не падает с ошибкой. После преобразования проверьте, сколько новых пропусков появилось: `df["col"].isna().sum()`. Если их много — где-то проблема в исходных данных.

4.4. Обработка дат

Даты требуют особого внимания. Формат может быть разным (`YYYY-MM-DD`, `DD.MM.YYYY`, `MM/DD/YYYY`), а часовые пояса добавляют отдельный уровень сложности. Pandas предоставляет `to_datetime()` с гибким параметром `format` для явного указания формата. После преобразования открываются возможности: извлечение компонент (год, месяц, день недели), расчёт разницы, скользящие окна.

Jupyter Notebook

```
# Преобразование с автоматическим определением формата
df_work["date_dt"] = pd.to_datetime(df_work["order_date"], errors="coerce")

# Явное указание формата — быстрее и точнее
df_work["date_dt"] = pd.to_datetime(df_work["order_date"], format="%Y-%m-%d")

# Извлечение компонент
df_work["year"] = df_work["date_dt"].dt.year
df_work["month"] = df_work["date_dt"].dt.month
df_work["day"] = df_work["date_dt"].dt.day
df_work["weekday"] = df_work["date_dt"].dt.day_name()
df_work["is_weekend"] = df_work["date_dt"].dt.weekday >= 5

# Разница между датами
df_work["days_since_start"] = (df_work["date_dt"] - df_work["date_dt"].min()).dt.days

# Фильтрация по диапазону дат
mask_2024 = (df_work["date_dt"] >= "2024-01-01") & (df_work["date_dt"] <= "2024-12-31")
df_2024 = df_work[mask_2024]

# Проверка
print(df_work[["date_dt", "year", "month", "weekday", "is_weekend"]].head())
```

ПРИМЕЧАНИЕ. Формат даты и локаль

`%Y` — четырёхзначный год, `%m` — месяц (01-12), `%d` — день (01-31), `%H` — часы, `%M` — минуты, `%S` — секунды. Полный список — в документации Python `datetime.strftime`. Если даты на русском (например, "31 января 2024"), придётся использовать `format="%d %B %Y"` с учётом локали.

4.5. Очистка текстовых значений

Текстовые столбцы часто содержат мусор: лишние пробелы, разный регистр, скрытые символы, опечатки. Pandas через `.str` даёт доступ к строковым методам. Регулярная очистка текста перед агрегацией или объединением таблиц — обязательный шаг, иначе два "одинаковых" значения окажутся разными.

Jupyter Notebook

```
# Удаление пробелов в начале и конце
df_work["client_clean"] = df_work["client"].str.strip()

# Приведение к нижнему регистру
df_work["client_lower"] = df_work["client"].str.lower()

# Замена внутренних пробелов на одно
df_work["client_norm"] = df_work["client"].str.replace(r"\s+", " ",
regex=True).str.strip()

# Замена значения
df_work["channel_clean"] = df_work["channel"].str.replace("Online", "WEB")

# Заполнение пропусков в строковом столбце
df_work["client_filled"] = df_work["client"].fillna("Unknown")

# Проверка: как выглядят категории после очистки
print(df_work["client_clean"].value_counts(dropna=False))
```

СОВЕТ. str.strip() перед value_counts

Перед `value_counts()` всегда делайте `str.strip()` — это объединит "Books" и "Books " в одну категорию. Для ещё большей надёжности добавьте `str.lower()` или `str.title()`, чтобы привести все варианты к одному регистру.

4.6. Фильтрация и сортировка с проверкой

Фильтрация выбирает строки по условию, сортировка меняет порядок. Обе операции не должны изменять сами значения, но влияют на индекс и порядок. После каждой фильтрации полезно проверять: сколько строк осталось, не потеряли ли мы что-то важное, как изменился диапазон значений.

Jupyter Notebook

```
# Фильтрация
df_high = sales_df[sales_df["amount"] > 100]
df_moscow = sales_df[sales_df["region"] == "Moscow"]
df_combined = sales_df[(sales_df["amount"] > 100) & (sales_df["region"] == "Moscow")]

# Проверка
print("Было:", len(sales_df))
print("Осталось:", len(df_combined))
print("Доля:", len(df_combined) / len(sales_df))

# Сортировка
df_sorted = df_combined.sort_values("amount", ascending=False)

# Проверка сортировки
print("Максимум:", df_sorted["amount"].max())
print("Первое значение:", df_sorted["amount"].iloc[0])

# Сброс индекса после фильтрации
df_reset = df_sorted.reset_index(drop=True)
print(df_reset.head())
```



Фильтр оставляет строки с amount > 100, затем reset_index даёт чистый индекс

ПРИМЕЧАНИЕ. reset_index(drop=True)

После фильтрации индекс сохраняет "дыры" от удалённых строк. Часто удобнее сбросить индекс, чтобы он снова стал 0, 1, 2, ... `drop=True` отбрасывает старый индекс, а не добавляет его как столбец.

4.7. Поиск выбросов

Выбросы — значения, значительно открывающиеся от основной массы. Они могут быть как ошибками данных, так и реальными аномалиями. Для поиска используют IQR (межквартильный размах), z-оценку или простой порог по перцентилю. После нахождения нужно решить: удалить, заменить на границу, оставить с пометкой.

Jupyter Notebook

```
# IQR-метод
q1 = df_work["amount"].quantile(0.25)
q3 = df_work["amount"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

print(f"IQR границы: [{lower:.2f}, {upper:.2f}]")

# Маска выбросов
mask_outliers = (df_work["amount"] < lower) | (df_work["amount"] > upper)
print("Выбросов:", mask_outliers.sum())

# Удаление выбросов
df_no_outliers = df_work[~mask_outliers]

# Ограничение выбросов (clip)
df_clipped = df_work.copy()
df_clipped["amount"] = df_clipped["amount"].clip(lower=lower, upper=upper)

# Перцентильный порог (например, удалить верхние 1%)
threshold = df_work["amount"].quantile(0.99)
df_below_99 = df_work[df_work["amount"] <= threshold]
```

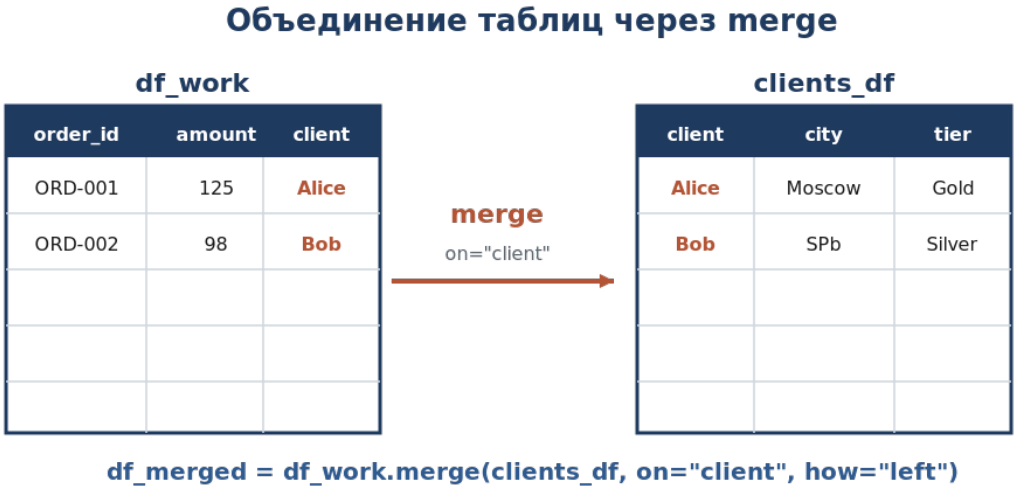
ВНИМАНИЕ. Не удаляйте выбросы автоматически

Выброс — не всегда ошибка. В финансовых данных это могут быть крупные сделки, в логах — аномальные события. Сначала изучите природу выброса, потом решайте. Удаление всех "хвостов" может убрать из данных самую интересную часть.



4.8. Объединение наборов данных

Объединение таблиц — ежедневная задача аналитика. Pandas предоставляет `merge()` (по ключам, как JOIN в SQL), `concat()` (по строкам или столбцам), `join()` (по индексу). Важно понимать тип соединения: `inner` (только совпадающие ключи), `outer` (все ключи), `left`/`right` (приоритет одной таблице).



Jupyter Notebook

```
# Создадим второй набор — справочник клиентов
clients_df = pd.DataFrame({
"client": ["Alice", "Bob", "Charlie", "Diana", "Eve"],
"city": ["Moscow", "Saint Petersburg", "Kazan", "Novosibirsk", "Moscow"],
"tier": ["Gold", "Silver", "Silver", "Bronze", "Gold"],
})

# merge по ключу
df_merged = df_work.merge(clients_df, on="client", how="left")

# merge по разным именам столбцов
df_merged2 = df_work.merge(
clients_df.rename(columns={"client": "customer"}),
left_on="client",
right_on="customer",
how="left",
)

# Проверка
print("Было:", df_work.shape, "Стало:", df_merged.shape)
print("Новые столбцы:", set(df_merged.columns) - set(df_work.columns))
print("Пропуски в новых столбцах:")
print(df_merged[["city", "tier"]].isna().sum())

# concat по строкам
df_more = pd.concat([sales_df, sales_df], ignore_index=True)
print("Дубликаты после concat:", df_more.duplicated().sum())

# concat по столбцам
df_side_by_side = pd.concat([sales_df.reset_index(drop=True),
clients_df.head(len(sales_df))], axis=1)
```

СОВЕТ. Проверяйте размер после merge

После `merge` число строк может неожиданно вырасти — если в правой таблице есть несколько записей с тем же ключом, произойдёт "размножение" строк левой. Всегда проверяйте `df.shape` до и после. Если число строк изменилось непредсказуемо — изучите дубликаты в ключах правой таблицы: `right_df["key"].value_counts()`.

4.9. Агрегации и groupby

`groupby` — основной инструмент агрегации в pandas. Группируем по одному или нескольким столбцам, применяем агрегатные функции. Полезные функции: `sum`, `mean`, `median`, `min`, `max`, `count`, `nunique`, `std`, `var`. `agg()` позволяет применить несколько функций сразу к разным столбцам.

Jupyter Notebook

```
# Простая агрегация по одной колонке
df_work.groupby("region")["amount"].sum()

# Несколько функций к одному столбцу
df_work.groupby("region")["amount"].agg(["sum", "mean", "count"])

# Разные функции к разным столбцам
df_work.groupby("region").agg({
"amount": ["sum", "mean"],
"quantity": "sum",
"order_id": "count",
})

# Группировка по нескольким столбцам
df_work.groupby(["region", "category"])["amount"].sum()

# Переименование результатов агрегации
df_work.groupby("region").agg(
total_amount=("amount", "sum"),
avg_amount=("amount", "mean"),
n_orders=("order_id", "count"),
)

# reset_index после groupby — чтобы превратить индекс в столбцы
agg_result = df_work.groupby(["region", "category"], as_index=False)["amount"].sum()
```

ПРИМЕЧАНИЕ. as_index=False — группировка без индекса

По умолчанию `groupby` превращает группирующий столбец в индекс. Если вам нужен обычный DataFrame со столбцами, передайте `as_index=False` или вызовите `reset_index()` после. Это удобнее для дальнейшей фильтрации и визуализации.

4.10. Сводные таблицы

`pivot_table` — мощный инструмент перекрёстной агрегации. Создаёт матрицу: строки — одни категории, столбцы — другие, ячейки — агрегированные значения. Полезен для отчётов, табличной визуализации и быстрого сравнения групп.

Jupyter Notebook

```
# Сводная таблица: регион × категория, сумма amount
pivot_sum = df_work.pivot_table(
values="amount",
index="region",
columns="category",
aggfunc="sum",
fill_value=0,
)

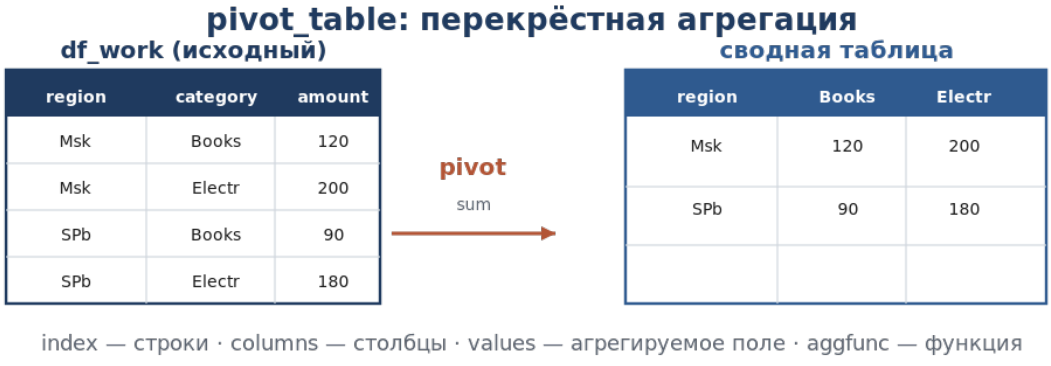
# Несколько агрегатных функций
pivot_multi = df_work.pivot_table(
values="amount",
index="region",
columns="category",
aggfunc=["sum", "mean"],
)

# С margins — итоги по строкам и столбцам
pivot_margins = df_work.pivot_table(
values="amount",
index="region",
columns="category",
aggfunc="sum",
margins=True,
margins_name="Итого",
)

print(pivot_margins)
```

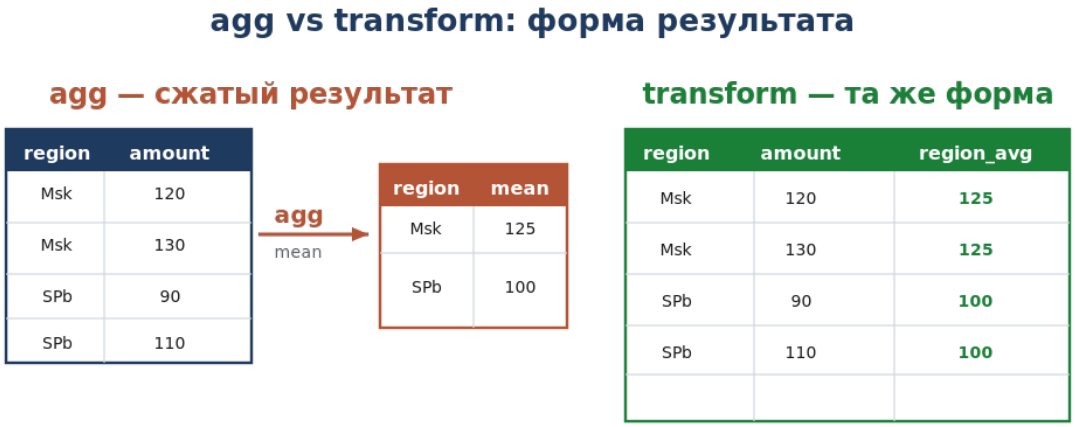
СОВЕТ. pivot_table vs groupby

`pivot_table` удобен, когда нужен результат в виде "матрицы" для отчёта или тепловой карты. `groupby` лучше, когда результат пойдёт в дальнейший код или визуализацию через длинный формат. Внутренне они оба работают через groupby + unstack.



4.11. transform: агрегация с сохранением формы

`transform` возвращает Series той же длины, что и исходный DataFrame. Это удобно, когда нужно добавить агрегированное значение обратно в каждую строку. Например, добавить средний чек по региону в каждую строку заказа.



Jupyter Notebook

```
# Средний чек по региону в каждой строке
df_work["region_avg"] = df_work.groupby("region")["amount"].transform("mean")

# Доля суммы заказа от суммы по региону
df_work["share_of_region"] = df_work["amount"] /
df_work.groupby("region")["amount"].transform("sum")

# Ранг по сумме внутри каждого региона
df_work["rank_in_region"] = df_work.groupby("region")["amount"].rank(
method="dense", ascending=False
)

# Проверка
print(df_work[["order_id", "region", "amount", "region_avg", "share_of_region"]].head())
```

ПРИМЕЧАНИЕ. transform vs agg

`agg` возвращает сжатый результат — одну строку на группу. `transform` возвращает Series с тем же индексом и длиной, что и исходный DataFrame. Используйте `transform`, когда нужно добавить агрегат в каждую строку, и `agg`, когда нужен итог по группам.

4.12. value_counts как базовый аналитический инструмент

`value_counts()` — одна из самых часто используемых команд. Она считает частоты уникальных значений и возвращает Series, отсортированный по убыванию частоты. Подходит для быстрой проверки категорий, поиска редких значений, анализа распределений.

Jupyter Notebook

```
# Базовый вызов
print(df_work["region"].value_counts())

# С долями
print(df_work["region"].value_counts(normalize=True).round(3))

# С пропусками
print(df_work["client"].value_counts(dropna=False))

# Топ-3
print(df_work["category"].value_counts().head(3))

# Без сортировки (в порядке появления значений)
print(df_work["region"].value_counts(sort=False))

# Бины для числового столбца
print(df_work["amount"].value_counts(bins=5))
```

СОВЕТ. value_counts(bins=N) — быстрый гистограммный анализ

Параметр `bins=N` разбивает числовой столбец на N равных интервалов и считает число значений в каждом. Это быстрый способ увидеть распределение без построения графика: `df["amount"].value_counts(bins=10)`.

5. Целостность и безопасность данных

Самая частая причина потери рабочих часов — не ошибка в коде, а незаметное изменение исходных данных. Аналитик открывает ноутбук на следующий день, повторно запускает ячейки и обнаруживает, что `df_raw` уже не тот. Поэтому в этом разделе мы разбираем дисциплину работы: исходный набор не трогаем, все изменения — в рабочей копии, промежуточные результаты сохраняем, контрольные суммы и размеры проверяем до и после каждого преобразования.

5.1. Дисциплина df_raw, df_work, df_clean

В этом материале мы договорились о трёх переменных. `df_raw` — то, что мы получили из источника (файла, базы, API). Его мы не меняем. `df_work` — рабочая копия, в которой выполняем очистку, преобразование, агрегацию. `df_clean` — финальный набор, который пойдёт в визуализацию и сохранение. Такая структура помогает вернуться к исходным данным в любой момент и сравнить результат с тем, что было.

Jupyter Notebook

```
import pandas as pd

# df_raw — исходный набор, не трогаем
df_raw = sales_df.copy()
print("df_raw shape:", df_raw.shape)
print("df_raw dtypes:\n", df_raw.dtypes)

# df_work — рабочая копия для очистки
df_work = df_raw.copy()

# Любое изменение — на df_work
df_work["amount"] = df_work["amount"].fillna(df_work["amount"].median())
df_work = df_work.drop_duplicates()

# Проверка: df_raw не изменился
print("df_raw shape после очистки:", df_raw.shape)
print("df_work shape после очистки:", df_work.shape)

# df_clean — финальный набор для визуализации и сохранения
df_clean = df_work.copy()
```

ВНИМАНИЕ. copy() — обязательно

Присваивание `df_work = df_raw` НЕ создаёт копию — это просто второе имя для того же объекта. Любое изменение `df_work` изменит и `df_raw`. Всегда используйте `df_raw.copy()` для создания рабочей версии. Если вы редактируете столбец и видите `SettingWithCopyWarning` — это сигнал, что что-то не так с копией.

5.2. Риски незаметного изменения

pandas во многих случаях возвращает представление (view), а не копию. Это значит, что изменение через одно имя может затронуть и другое. Особенно коварны следующие ситуации: выбор столбца через `df[["a"]]` возвращает копию, а `df["a"]` возвращает Series, который может быть view. Фильтрация `df[df["a"] > 0]` возвращает копию, но `df.loc[df["a"] > 0, "b"]` — может быть view на исходник.

Jupyter Notebook

```
# Опасный пример: цепная индексация
# df_work[df_work["amount"] > 100]["amount"] = 999 # SettingWithCopyWarning

# Безопасно: loc
df_work.loc[df_work["amount"] > 100, "amount"] = 999

# Опасный пример: slice без .copy()
subset = df_work[df_work["region"] == "Moscow"] # view, не копия
# subset["new_col"] = 1 # Warning!

# Безопасно: .copy() после фильтрации
subset = df_work[df_work["region"] == "Moscow"].copy()
subset["new_col"] = 1 # OK
```

СОВЕТ. Правило копии после фильтрации

После любой фильтрации, которая возвращает DataFrame, всегда добавляйте `.copy()`, если планируете изменять результат. Это гарантированно создаёт новый объект, и вы не получите `SettingWithCopyWarning`. Накладные расходы по памяти — минимальные, а безопасность — существенная.

5.3. Контрольные итоги до и после преобразования

Перед любым "разрушающим" преобразованием (удаление строк, изменение типов, агрегация) полезно зафиксировать контрольные показатели: число строк, сумма ключевого числового столбца, число пропусков. После преобразования сравниваем. Если сумма изменилась на 30% — что-то пошло не так, хотя код не выдал ошибку.

Jupyter Notebook

```
# Фиксация контрольных показателей
def snapshot(df: pd.DataFrame, name: str = "") -> dict:
'''Снимок состояния DataFrame для последующего сравнения.'''
return {
"name": name,
"rows": len(df),
"cols": df.shape[1],
"amount_sum": df["amount"].sum() if "amount" in df.columns else None,
"amount_mean": df["amount"].mean() if "amount" in df.columns else None,
"n_na_amount": df["amount"].isna().sum() if "amount" in df.columns else None,
}

before = snapshot(df_work, "before")
print("Before:", before)

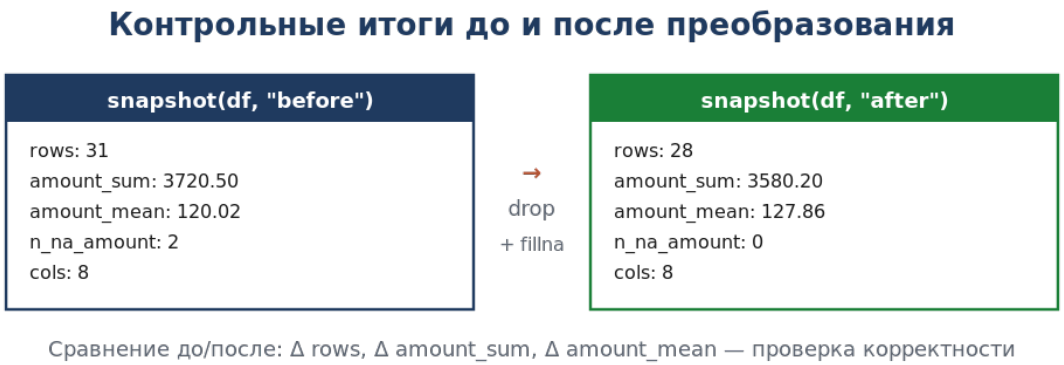
# Выполняем преобразование
df_work = df_work.dropna(subset=["amount"])
df_work = df_work.drop_duplicates()

after = snapshot(df_work, "after")
print("After:", after)

# Сравнение
print("Δ rows:", after["rows"] - before["rows"])
print("Δ amount_sum:", round(after["amount_sum"] - before["amount_sum"], 2))
print("Δ amount_mean:", round(after["amount_mean"] - before["amount_mean"], 2))
```

ПРИМЕЧАНИЕ. Что должно совпадать

При удалении дубликатов и пропусков строки исчезают, но сумма amount в оставшихся строках должна быть близка к исходной (уменьшится ровно на сумму удалённых строк). Если сумма упала сильно больше, чем число удалённых строк × среднее — где-то ошибка в логике.



Сравнение до/после: Δ rows, Δ amount_sum, Δ amount_mean — проверка корректности

5.4. Резервные копии и промежуточные срезы

Перед серьёзным шагом (объединение с другой таблицей, агрегация с потерей строк, перезапись столбца) полезно сделать промежуточную копию. Если что-то пойдёт не так, не придётся начинать сначала — достаточно вернуться к `df_work_backup`. Это особенно важно при работе в Jupyter, где ячейки можно запустить в любом порядке, и состояние переменных непредсказуемо.

Jupyter Notebook

```
# Резервная копия перед опасным шагом
df_work_backup = df_work.copy()

# Опасный шаг: merge с другой таблицей
df_work = df_work.merge(clients_df, on="client", how="left")

# Проверка
print("До merge:", df_work_backup.shape)
print("После merge:", df_work.shape)

# Если что-то не так — откат
if df_work.shape[0] != df_work_backup.shape[0]:
print("Размер изменился! Проверяю дубликаты в ключе.")
print(clients_df["client"].value_counts().head())
# Откат при необходимости:
# df_work = df_work_backup.copy()
```

СОВЕТ. Резервные копии в одну строку

Создавайте резервные копии систематически: `df_bak_1 = df.copy()` перед первым шагом, `df_bak_2 = df.copy()` перед вторым. Это дешёвая страховка — память освобождается, как только вы удаляете переменную через `del df_bak_1`.

5.5. Сохранение исходных данных в неизменном виде

Самая надёжная защита — физически хранить исходный файл в режиме "только для чтения" и никогда не перезаписывать его результатами. При сохранении результатов используйте другое имя файла, а не имя исходника. Для критичных данных можно сделать несколько поколений копий: `sales_v1.csv`, `sales_v2.csv`, и т.д.

Jupyter Notebook

```
# Чтение исходника
df_raw = pd.read_csv("data/sales.csv")

# Сохранение результата — НИКОГДА не под тем же именем!
df_clean.to_csv("data/sales_clean.csv", index=False)

# С версионированием
from datetime import datetime
version = datetime.now().strftime("%Y%m%d_%H%M")
df_clean.to_csv(f"data/sales_clean_{version}.csv", index=False)

# Проверка: исходник не тронут
df_raw_again = pd.read_csv("data/sales.csv")
print("Исходник не изменился:", df_raw.equals(df_raw_again))
```

ВНИМАНИЕ. Никогда не перезаписывайте исходный файл

`df_raw.to_csv("data/sales.csv")` перезапишет исходник. Если вы потом поймёте, что в очистке была ошибка — восстановить данные не получится. Сохраняйте результаты под другими именами: `sales_clean.csv`, `sales_processed.csv`, `sales_v2.csv`.

5.6. Проверка размеров, типов, пропусков — три уровня контроля

Контроль должен быть трёхуровневым: до (фиксируем исходное состояние), во время (после каждого важного шага), после (финальная проверка). Такой подход превращает "магию" в воспроизводимый процесс. Если что-то пойдёт не так, вы всегда сможете сказать, на каком шаге это произошло.

Jupyter Notebook

```
def audit(df: pd.DataFrame, step_name: str) -> None:
'''Контроль состояния DataFrame после шага.'''
print(f"\n=== {step_name} ===")
print(f"shape: {df.shape}")
print(f"dtypes:\n{df.dtypes}")
print(f"na sum:\n{df.isna().sum()}")
print(f"memory (MB): {df.memory_usage(deep=True).sum() / 1e6:.2f}")

audit(df_raw, "Шаг 0: исходный")

df_work = df_raw.copy()
df_work["amount"] = df_work["amount"].fillna(df_work["amount"].median())
audit(df_work, "Шаг 1: заполнение пропусков в amount")

df_work = df_work.drop_duplicates()
audit(df_work, "Шаг 2: удаление дубликатов")

df_work = df_work.dropna(subset=["client"])
audit(df_work, "Шаг 3: удаление строк без клиента")
```

ПРИМЕЧАНИЕ. Аудит в виде функции

Функция `audit` — отличный кандидат на добавление в `utils.py`. Она выводит ключевую информацию за один вызов и экономит время на диагностике. Можно расширить её: добавить проверку уникальности ключей, диапазонов числовых столбцов, частот категорий.

5.7. Защита от потери типов и порядка

Несколько типичных "подводных камней" при экспорте и импорте. CSV не сохраняет типы — все данные становятся строками при чтении. После `to_csv()` и нового `read_csv()` числовой столбец может оказаться `object`. Excel сохраняет типы в ячейках, но `read_excel` иногда присваивает `object` вместо `int64`. Parquet сохраняет типы полностью — это его главное преимущество.



CSV теряет типы при round-trip

Parquet сохраняет типы полностью

После чтения CSV проверяйте dtypes: pd.to_datetime, pd.to_numeric

Jupyter Notebook

```
# Сценарий: сохранили в CSV и прочитали обратно
df_work.to_csv("data/temp.csv", index=False)
df_back = pd.read_csv("data/temp.csv")

# Сравнение типов
print("Исходные типы:\n", df_work.dtypes)
print("После CSV round-trip:\n", df_back.dtypes)

# Parquet: типы сохраняются
df_work.to_parquet("data/temp.parquet")
df_back_pq = pd.read_parquet("data/temp.parquet")
print("После Parquet round-trip:\n", df_back_pq.dtypes)
print("Equal dtypes:", df_work.dtypes.equals(df_back_pq.dtypes))
```

ВНИМАНИЕ. CSV round-trip меняет типы

Если в числовом столбце был `int64`, после `to_csv` + `read_csv` он может стать `int64` снова, если нет пропусков. Но если есть NaN — станет `float64`. И `datetime` после CSV всегда становится строкой. После чтения CSV обязательно повторно проверяйте типы и при необходимости приводите: `pd.to_datetime`, `pd.to_numeric`.

5.8. Контрольный пример: полный сценарий очистки

Соберём все принципы в один сквозной сценарий. Исходник — `df_raw`. Рабочая копия — `df_work`. Аудит — на каждом шаге. Финал — `df_clean`, который готов к визуализации и сохранению. Этот блок можно скопировать в реальный проект и адаптировать под свои столбцы.

Jupyter Notebook

```
# 1. Исходник
df_raw = sales_df.copy()
audit(df_raw, "Шаг 0: исходный")

# 2. Рабочая копия
df_work = df_raw.copy()

# 3. Удаление дубликатов
df_work = df_work.drop_duplicates()
audit(df_work, "Шаг 1: drop_duplicates")

# 4. Заполнение пропусков
df_work["amount"] = df_work["amount"].fillna(df_work["amount"].median())
df_work["client"] = df_work["client"].fillna("Unknown")
df_work["quantity"] = df_work["quantity"].fillna(1).astype(int)
audit(df_work, "Шаг 2: fillna")

# 5. Преобразование даты
df_work["order_date"] = pd.to_datetime(df_work["order_date"])
audit(df_work, "Шаг 3: to_datetime")

# 6. Финальный набор
df_clean = df_work.copy()
audit(df_clean, "Шаг 4: df_clean")

print("\n=== ИТОГО ===")
print(f"Исходник: {df_raw.shape}, Чистый: {df_clean.shape}")
```

СОВЕТ. Этот блок — шаблон для всех проектов

Скопируйте этот сценарий в свой проект, замените имена столбцов на свои — и вы получите воспроизводимую очистку с аудитом. Это особенно полезно, когда данные обновляются регулярно и нужно убедиться, что новая порция прошла все проверки.

6. Визуализация

Визуализация — это финальный шаг аналитического сценария, на котором данные становятся понятными без чтения таблиц. Графики помогают заметить выбросы, увидеть тренды, сравнить группы. Мы используем matplotlib и seaborn: первый даёт полный контроль, второй упрощает статистические графики. В этом разделе — пять ключевых типов графиков, которые покрывают 90% задач аналитика: столбчатая диаграмма, гистограмма, boxplot, линейный график и тепловая карта корреляций.

6.1. Подготовка данных и общий шаблон

Перед построением графика данные нужно подготовить: выбрать нужный срез, агрегировать, привести к длинному или широкому формату. Не пытайтесь построить график прямо из "сырого" DataFrame с тысячами строк — сначала агрегируйте до 10-50 значений. Также важно настроить стиль: размер фигуры, шрифт, тему. Эти настройки можно сделать один раз в начале ноутбука.

Jupyter Notebook

```
import matplotlib.pyplot as plt
import seaborn as sns

# Настройка стиля для всего ноутбука
sns.set_theme(style="whitegrid", palette="muted")
plt.rcParams["figure.figsize"] = (10, 6)
plt.rcParams["font.size"] = 11

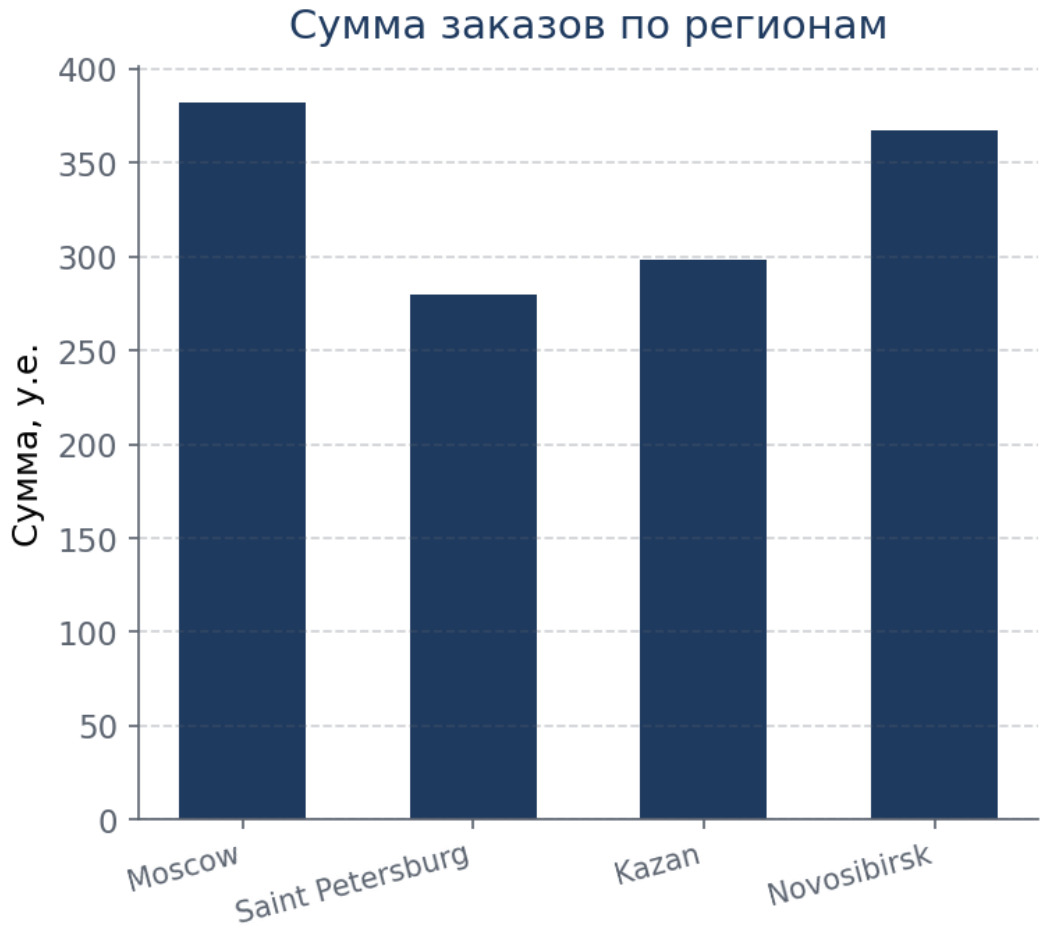
# Подготовка данных для визуализации
df_viz = df_clean.copy()
print(df_viz.shape)
df_viz.head()
```

СОВЕТ. Один стиль на весь ноутбук

Вызов `sns.set_theme()` в начале ноутбука задаёт единый стиль для всех графиков. Не нужно повторять настройки в каждой ячейке. Параметр `palette="muted"` даёт спокойные, профессиональные цвета.

6.2. Столбчатая диаграмма

Столбчатая диаграмма показывает агрегированные значения по категориям. Подходит для сравнения суммы или среднего по группам: выручка по регионам, число заказов по категориям. В seaborn используется `barplot()` (агрегирует данные) или `countplot()` (считает частоты).



Jupyter Notebook

```
# Подготовка: сумма заказов по регионам
region_sum = df_viz.groupby("region")["amount"].sum().reset_index()

# Столбчатая диаграмма
fig, ax = plt.subplots(figsize=(10, 6))
sns.barplot(data=region_sum, x="region", y="amount", ax=ax, color="#1F3A5F")
ax.set_title("Сумма заказов по регионам", fontsize=14)
ax.set_xlabel("Регион")
ax.set_ylabel("Сумма, у.е.")
ax.tick_params(axis="x", rotation=20)

# Сохранение
fig.savefig("figures/bar_region.png", dpi=150, bbox_inches="tight")
plt.show()
```

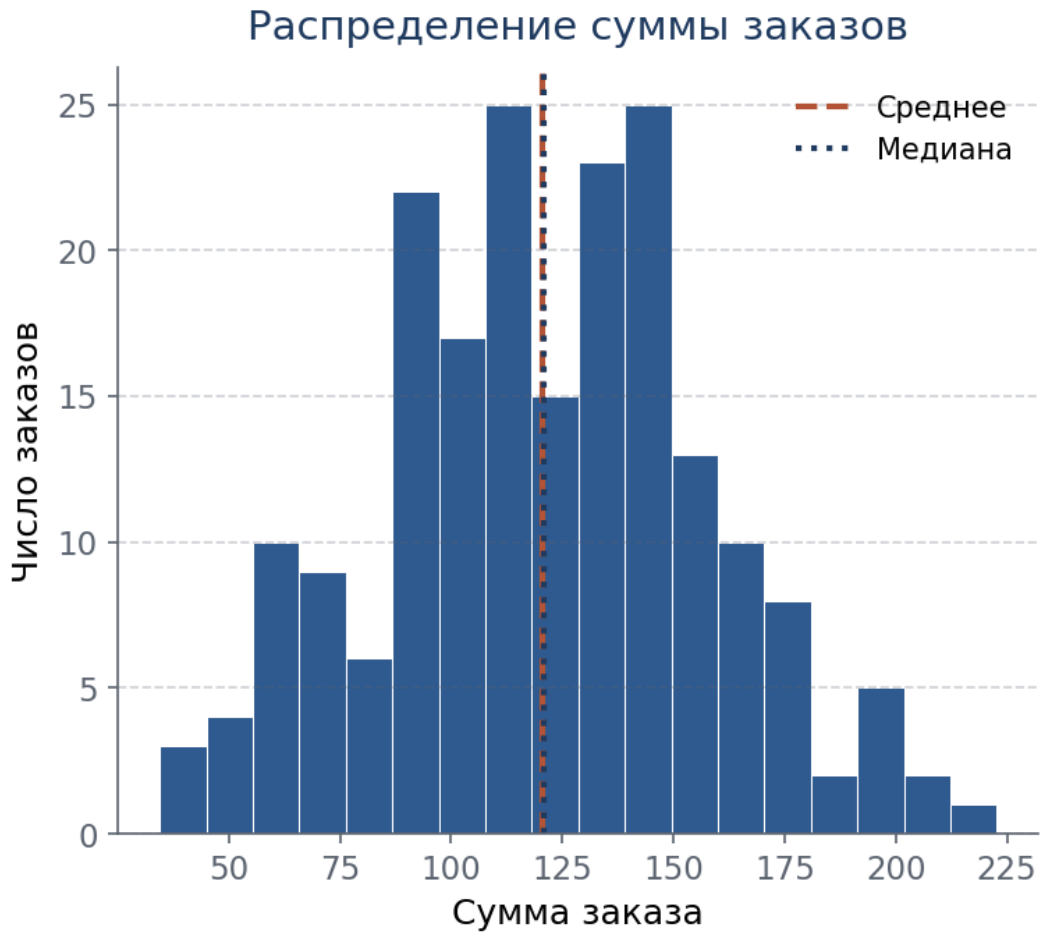
СОВЕТ. color вместо palette для одного цвета

В новых версиях seaborn для однотонных графиков нужно передавать `color`, а не `palette`. Если вы используете `palette` с одним значением, появится предупреждение. Для нескольких категорий используйте `hue` и `palette`.



6.3. Гистограмма

Гистограмма показывает распределение числового признака. Подходит для оценки формы распределения: симметричное, скошенное, с выбросами. Параметр `bins` управляет числом корзин: слишком мало — потеряем детали, слишком много — будет шумно. Хорошее правило — `bins=30` для небольших данных, `bins="auto"` для автоматического подбора.



Jupyter Notebook

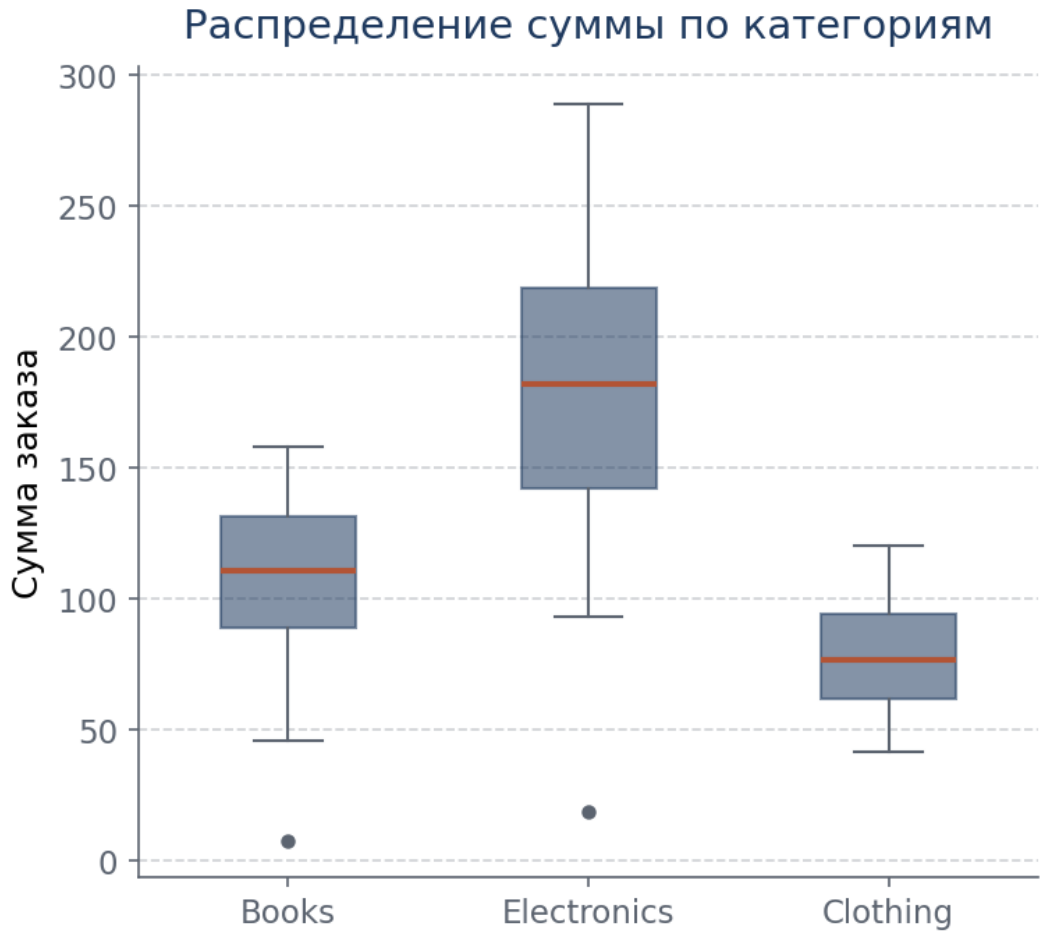
```
fig, ax = plt.subplots(figsize=(10, 6))
sns.histplot(data=df_viz, x="amount", bins=20, kde=True, color="#2F5A8F", ax=ax)
ax.set_title("Распределение суммы заказов", fontsize=14)
ax.set_xlabel("Сумма заказа")
ax.set_ylabel("Число заказов")
ax.axvline(df_viz["amount"].mean(), color="red", linestyle="--", label=f"Среднее:
{df_viz['amount'].mean():.1f}")
ax.axvline(df_viz["amount"].median(), color="green", linestyle="--", label=f"Медиана:
{df_viz['amount'].median():.1f}")
ax.legend()
fig.savefig("figures/hist_amount.png", dpi=150, bbox_inches="tight")
plt.show()
```

ПРИМЕЧАНИЕ. kde=True добавляет сглаженную кривую

Параметр `kde=True` накладывает поверх гистограммы ядерную оценку плотности — сглаженную кривую распределения. Это помогает визуально оценить форму, не отвлекаясь на шум от корзин.

6.4. Boxplot

Boxplot (ящик с усами) показывает медиану, квартили, размах и выбросы по группам. Незаменим для сравнения распределений нескольких категорий: видны медианы, размахи, аномалии. Boxplot "молчит" о форме распределения внутри ящика, но отлично показывает выбросы.



Jupyter Notebook

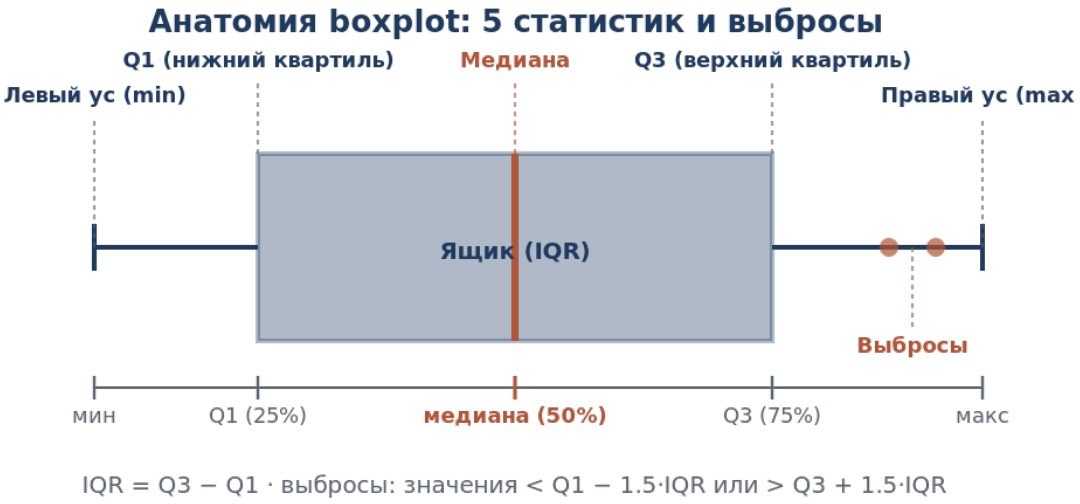
```
fig, ax = plt.subplots(figsize=(10, 6))
sns.boxplot(data=df_viz, x="category", y="amount", ax=ax, color="#1F3A5F")
ax.set_title("Распределение суммы по категориям", fontsize=14)
ax.set_xlabel("Категория")
ax.set_ylabel("Сумма заказа")

# Подписи medians на графике
medians = df_viz.groupby("category")["amount"].median()
for i, cat in enumerate(medians.index):
ax.text(i, medians[cat], f"{medians[cat]:.0f}", ha="center", va="bottom", fontsize=10,
color="red")

fig.savefig("figures/boxplot_category.png", dpi=150, bbox_inches="tight")
plt.show()
```

ВНИМАНИЕ. Boxplot не показывает размер выборки

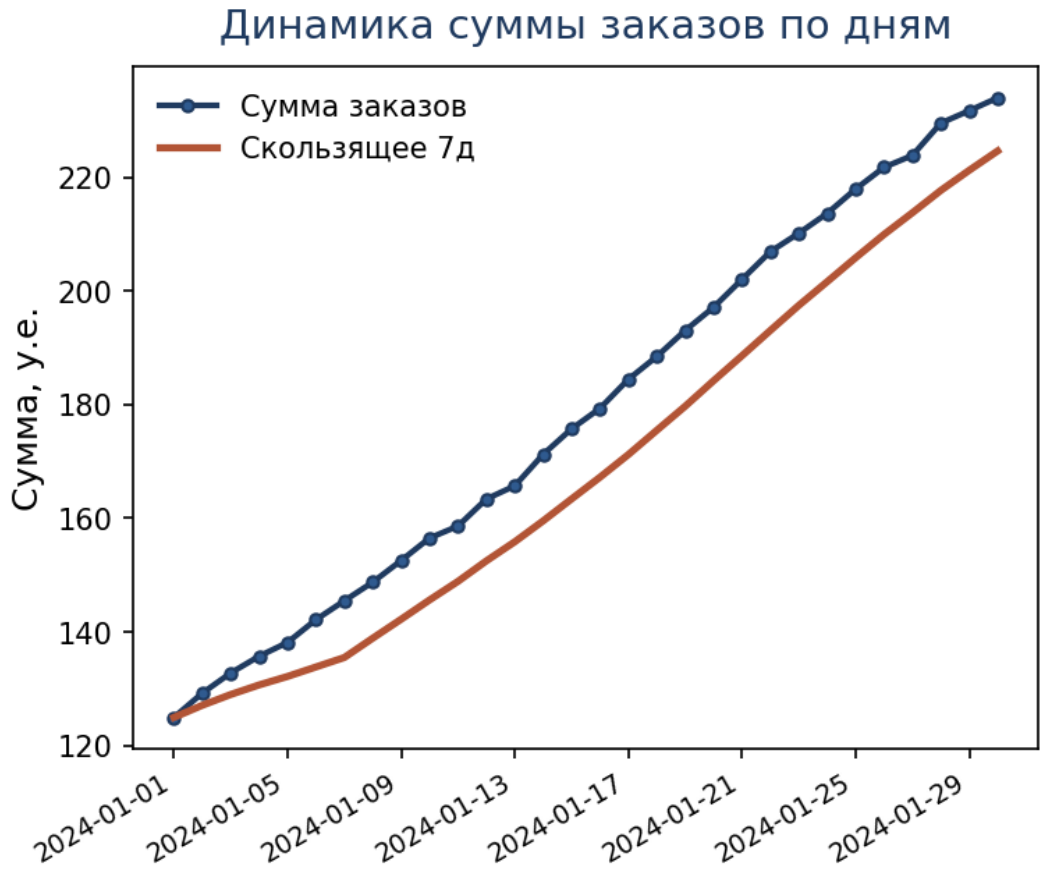
Если в одной категории 10 наблюдений, а в другой — 1000, boxplot выглядит одинаково. Для честного сравнения добавьте `sns.stripplot()` поверх `boxplot()`, чтобы показать отдельные точки, или используйте `sns.violinplot()` для оценки плотности.



IQR = Q3 − Q1 · выбросы: значения < Q1 − 1.5·IQR или > Q3 + 1.5·IQR

6.5. Линейный график

Линейный график показывает изменение признака во времени. Подходит для временных рядов: выручка по дням, число заказов по месяцам, средний чек по неделям. Перед построением убедитесь, что данные отсортированы по дате и сгруппированы до нужной гранулярности (день, неделя, месяц).



Jupyter Notebook

```
# Подготовка: сумма по дням
df_viz["order_date"] = pd.to_datetime(df_viz["order_date"])
daily = df_viz.groupby("order_date")["amount"].sum().reset_index()

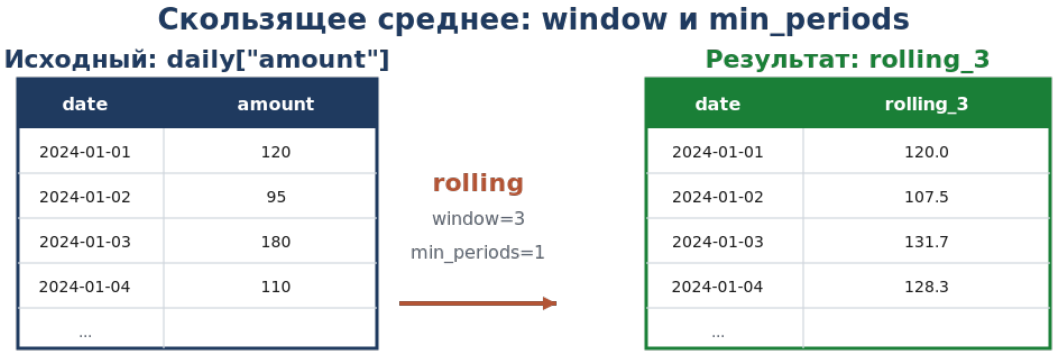
fig, ax = plt.subplots(figsize=(12, 6))
sns.lineplot(data=daily, x="order_date", y="amount", ax=ax, color="#1F3A5F", marker="o")
ax.set_title("Динамика суммы заказов по дням", fontsize=14)
ax.set_xlabel("Дата")
ax.set_ylabel("Сумма, у.е.")
ax.tick_params(axis="x", rotation=30)

# Добавим скользящее среднее
daily["rolling_7"] = daily["amount"].rolling(window=7, min_periods=1).mean()
ax.plot(daily["order_date"], daily["rolling_7"], color="#B25435", linewidth=2,
label="Скользящее 7д")
ax.legend()

fig.savefig("figures/line_daily.png", dpi=150, bbox_inches="tight")
plt.show()
```

СОВЕТ. Скользящее среднее для сглаживания

Скользящее среднее (`rolling(window=N).mean()`) сглаживает краткосрочные колебания и показывает тренд. Окно 7 — для дневных данных (неделя), 30 — для месячного сглаживания. `min_periods=1` позволяет считать среднее даже в начале, когда данных меньше окна.



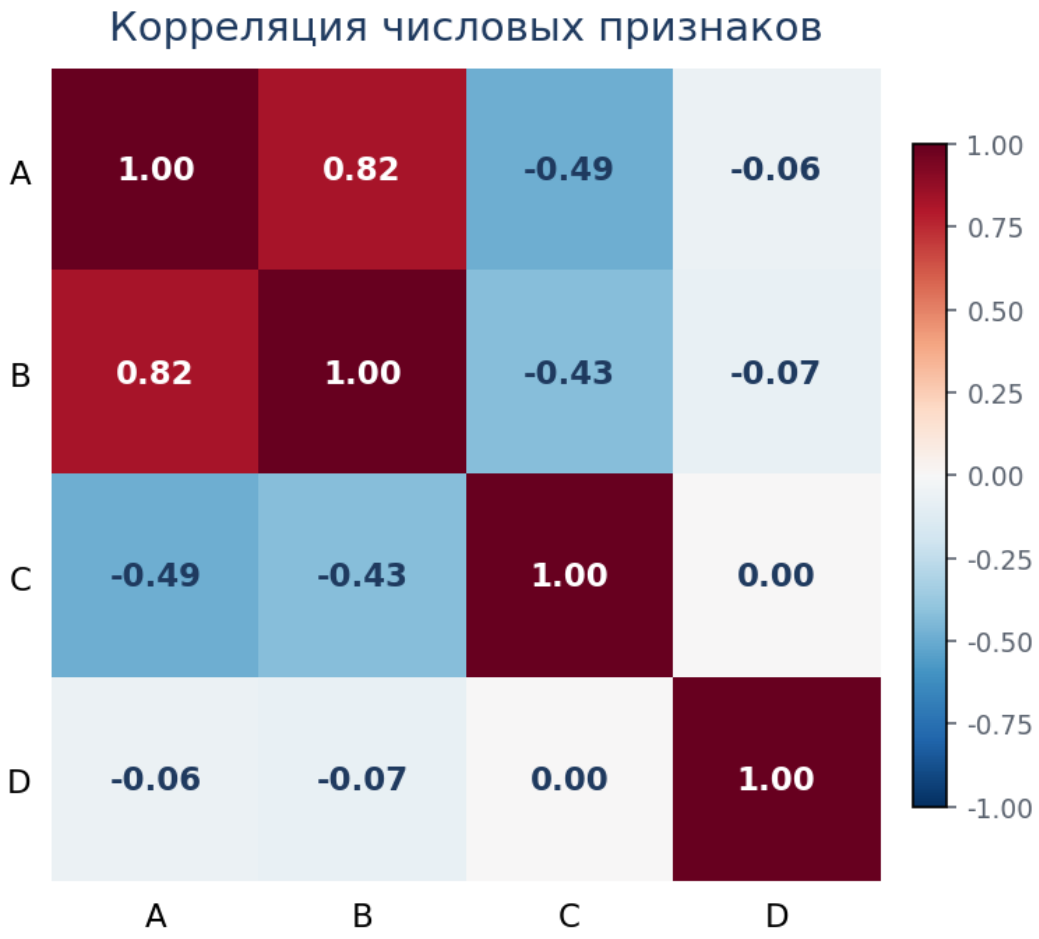
min_periods=1 — скользящее среднее считается с первой строки

Окно 7 — недельное сглаживание, 30 — месячное

rolling(window=N).mean() — основной вызов для сглаживания

6.6. Тепловая карта корреляций

Тепловая карта корреляций показывает, какие числовые признаки связаны друг с другом. Значения от -1 (обратная связь) до +1 (прямая). Подходит для разведочного анализа: поиск дублирующих признаков, потенциальных предикторов, скрытых связей.



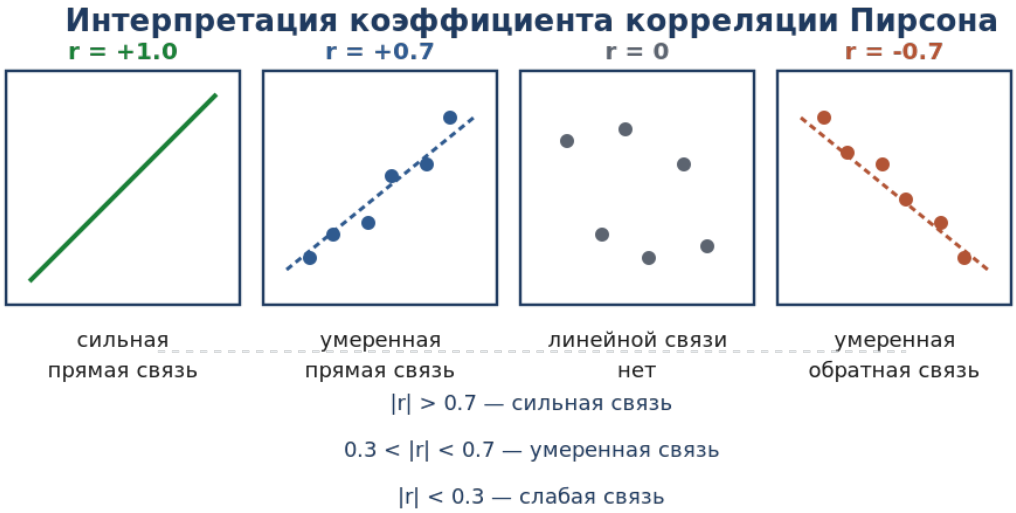
Jupyter Notebook

```
# Выбираем только числовые столбцы
numeric_df = df_viz.select_dtypes(include="number")
corr = numeric_df.corr()

fig, ax = plt.subplots(figsize=(10, 8))
sns.heatmap(
corr,
annot=True,
fmt=".2f",
cmap="RdBu_r",
center=0,
vmin=-1,
vmax=1,
square=True,
linewidths=0.5,
ax=ax,
)
ax.set_title("Корреляция числовых признаков", fontsize=14)
fig.savefig("figures/heatmap_corr.png", dpi=150, bbox_inches="tight")
plt.show()
```

ПРИМЕЧАНИЕ. Что показывает корреляция

Значения близко к +1 или -1 — сильная связь. Близко к 0 — связи нет. Но корреляция показывает только линейную связь: если признаки связаны нелинейно (например, U-образно), корреляция будет близка к нулю, хотя связь есть. Не интерпретируйте корреляцию как причинность.



6.7. Сохранение графика в файл

Каждый график стоит сохранять в файл, особенно если он пойдёт в отчёт или презентацию. Параметр `dpi=150` даёт хорошее качество при разумном размере файла. `bbox_inches="tight"` обрезает пустые поля. Для векторной графики используйте формат PDF или SVG — они масштабируются без потери качества.



Jupyter Notebook

```
# Сохранение в PNG
fig.savefig("figures/chart.png", dpi=150, bbox_inches="tight")

# Сохранение в PDF (векторный)
fig.savefig("figures/chart.pdf", bbox_inches="tight")

# Сохранение в SVG (векторный, веб)
fig.savefig("figures/chart.svg", bbox_inches="tight")

# Сохранение в JPG (для фото)
fig.savefig("figures/chart.jpg", dpi=150, bbox_inches="tight", quality=90)
```

СОВЕТ. Создавайте папку для фигур заранее

Перед сохранением убедитесь, что папка существует: `import os; os.makedirs("figures", exist_ok=True)`. Иначе `savefig` упадёт с `FileNotFoundError`. Это частая ошибка при первом запуске ноутбука на новой машине.

7. Сохранение результатов

После всех преобразований и анализа наступает этап сохранения результатов. Важно понимать, какой формат выбрать для какой задачи, какие параметры влияют на размер и качество файла, и как не перезаписать исходник случайно. В этом разделе — все ключевые параметры записи для Excel, CSV и Parquet, с примерами для локальной папки и Google Drive.

7.1. Сохранение в Excel

Для записи в Excel используется `to_excel()`. Базовый вызов прост, но параметры позволяют управлять именем листа, индексом, форматированием. Для записи в несколько листов нужно создать `ExcelWriter` — это позволяет записать несколько DataFrame в один файл.

Jupyter Notebook

```
# Базовое сохранение
df_clean.to_excel("data/sales_clean.xlsx", index=False)

# С именем листа
df_clean.to_excel("data/sales_clean.xlsx", sheet_name="Sales 2024", index=False)

# Без заголовков
df_clean.to_excel("data/sales_clean.xlsx", header=False, index=False)

# Запись нескольких DataFrame в один файл
with pd.ExcelWriter("data/sales_multi.xlsx") as writer:
df_clean.to_excel(writer, sheet_name="Clean", index=False)
df_work.to_excel(writer, sheet_name="Work", index=False)
clients_df.to_excel(writer, sheet_name="Clients", index=False)

# С явным движком
df_clean.to_excel("data/sales_clean.xlsx", engine="openpyxl", index=False)
```

ВНИМАНИЕ. index=False — почти всегда

По умолчанию `to_excel` записывает индекс как первый столбец. В 95% случаев это нежелательно — индекс просто дублирует нумерацию строк. Передавайте `index=False`, если только индекс не несёт смысловой нагрузки (например, это дата или идентификатор).

СОВЕТ. ExcelWriter с mode="a" — добавление листов

Если нужно добавить лист в существующий файл: `pd.ExcelWriter("data/file.xlsx", mode="a", engine="openpyxl")`. Без `mode="a"` файл будет перезаписан.

7.2. Сохранение в CSV

CSV — самый универсальный формат. Запись проста, но параметры сильно влияют на совместимость. Разделитель по умолчанию — запятая; для русского Excel используйте `sep=";"` и `encoding="utf-8-sig"` (с BOM, чтобы Excel корректно распознал кириллицу).

Jupyter Notebook

```
# Базовое сохранение
df_clean.to_csv("data/sales_clean.csv", index=False)

# С разделителем ; (для русского Excel)
df_clean.to_csv("data/sales_ru.csv", sep=";", index=False, encoding="utf-8-sig")

# Без заголовков
df_clean.to_csv("data/sales_noheader.csv", header=False, index=False)

# Только выбранные столбцы
df_clean.to_csv("data/sales_subset.csv", columns=["order_id", "amount"], index=False)

# С запятой как десятичным разделителем (для русского Excel)
df_clean.to_csv("data/sales_ru_dec.csv", sep=";", decimal=",", index=False,
encoding="utf-8-sig")

# Сжатый CSV (gzip)
df_clean.to_csv("data/sales.csv.gz", index=False, compression="gzip")
```

> **ПРИМЕЧАНИЕ. encoding="utf-8-sig" с BOM**
>
> `utf-8-sig` добавляет в начало файла три байта BOM (Byte Order Mark). Без них Excel на Windows может неверно определить кодировку и показать кракозябры. Для файлов, которые будут открываться в Excel на Windows, используйте `utf-8-sig`. Для других систем — обычный `utf-8`.

7.3. Сохранение в Parquet

Parquet оптимален для больших данных: компактный, с типами, с поддержкой сжатия. Запись через `to_parquet()` поддерживает несколько алгоритмов сжатия: `snappy` (по умолчанию, быстро), `gzip` (компактнее, медленнее), `brotli` (очень компактно, очень медленно). Выбор зависит от приоритета: скорость записи или размер файла.

Jupyter Notebook

```
# Базовое сохранение
df_clean.to_parquet("data/sales_clean.parquet")

# Без индекса
df_clean.to_parquet("data/sales_clean.parquet", index=False)

# Со сжатием gzip (компактнее)
df_clean.to_parquet("data/sales_gzip.parquet", compression="gzip", index=False)

# Со сжатием snappy (быстрее)
df_clean.to_parquet("data/sales_snappy.parquet", compression="snappy", index=False)

# Через явный движок
df_clean.to_parquet("data/sales_fp.parquet", engine="fastparquet", compression="gzip")
```

> **СОВЕТ. Сравните размеры файлов**
>
> После сохранения сравните размеры трёх форматов: `os.path.getsize("data/sales_clean.xlsx")`, `os.path.getsize("data/sales_clean.csv")`, `os.path.getsize("data/sales_clean.parquet")`. Для больших наборов Parquet обычно в 5-10 раз меньше CSV, и при этом сохраняет типы.

7.4. Сохранение в локальную папку, папку проекта и Google Drive

Путь сохранения зависит от среды. В локальном Jupyter — это путь относительно каталога ноутбука. В Colab — путь на примонтированном Google Drive. В PowerShell и Linux — это путь к целевому каталогу.

Jupyter Notebook

```
# Локальная папка
df_clean.to_csv("output/sales_clean.csv", index=False)

# Папка проекта (создаём, если нет)
import os
os.makedirs("output/clean", exist_ok=True)
df_clean.to_csv("output/clean/sales_2024.csv", index=False)
```

Google Colab

```
# Сохранение в Google Drive
df_clean.to_csv("/content/drive/MyDrive/projects/sales_clean.csv", index=False)

# Создание подпапки
import os
out_dir = "/content/drive/MyDrive/projects/clean"
os.makedirs(out_dir, exist_ok=True)
df_clean.to_csv(f"{out_dir}/sales_2024.csv", index=False)

# Сохранение в нескольких форматах
df_clean.to_excel(f"{out_dir}/sales_2024.xlsx", index=False)
df_clean.to_parquet(f"{out_dir}/sales_2024.parquet", index=False)
```

ВНИМАНИЕ. Colab без Drive — временные файлы

Если не примонтировать Drive, файлы сохранятся в `/content/`, который очищается при закрытии ноутбука. Для постоянного хранения используйте Drive.

7.5. Проверка сохранённого файла

После сохранения всегда перечитывайте файл и сравнивайте с тем, что сохраняли. Это простой, но критичный шаг: если файл записался с ошибкой (нет прав, нет места на диске, путь неверный), лучше узнать сразу.

Jupyter Notebook

```
import os

# Проверка, что файл существует и его размер
path = "data/sales_clean.csv"
print("Файл существует:", os.path.exists(path))
print("Размер, байт:", os.path.getsize(path))

# Перечитывание
df_check = pd.read_csv(path)

# Сравнение
print("Совпадение форм:", df_check.shape == df_clean.shape)
print("Совпадение столбцов:", list(df_check.columns) == list(df_clean.columns))
print("Число строк совпадает:", len(df_check) == len(df_clean))

# Полное сравнение (типы могут отличаться после CSV)
print("Типы до:", df_clean.dtypes.to_dict())
print("Типы после:", df_check.dtypes.to_dict())
```

ПРИМЕЧАНИЕ. Полное equals() работает не всегда

После CSV round-trip типы могут измениться: datetime становится object, int с NaN становится float. Поэтому `df_clean.equals(df_check)` вернёт False даже при правильных данных. Сравнивайте форму, столбцы и число строк, а типы — отдельно.

7.6. Какой формат выбрать

Выбор формата зависит от задачи. CSV — для обмена, небольших данных, ручного просмотра. Excel — для отчётов и обмена с пользователями Excel. Parquet — для аналитики, больших данных, машинного обучения. JSON — для API и веб-интеграции. SQLite — для локальной аналитической базы.

- CSV: универсальный обмен, чтение в любой среде, ручной просмотр. Минус — не сохраняет типы, большой размер.
- Excel: отчёты для коллег, ручное редактирование, несколько листов. Минус — медленно для больших данных.
- Parquet: хранение для аналитики, быстрые чтение/запись, сохранение типов. Минус — не открывается в Excel напрямую.
- JSON: API, веб-сервисы. Минус — плохо подходит для таблиц.
- SQLite: локальная БД, сложные запросы. Минус — нужно знать SQL.

> **СОВЕТ. Для аналитики — Parquet, для обмена — CSV**
>
> Если вы единственный пользователь данных и они большие — Parquet. Если файлы передаются в другую систему или другой команде — CSV с UTF-8. Excel — только когда получатель не умеет работать с другими форматами.

8. Работа в Jupyter, Colab, PowerShell и Linux

Каждая среда имеет свои особенности. В этом разделе — короткие рецепты для каждой: как запустить Jupyter, как клонировать проект, как активировать виртуальное окружение, как подключить Google Drive. Главное правило — не путать команды оболочки и Python-код: команды терминала в Jupyter должны предваряться `!`, а Python-код в PowerShell и Linux запускается через `python3`.

8.1. Jupyter Notebook

Jupyter — стандартная среда для аналитика. Запуск из терминала — `jupyter notebook` или `jupyter lab`. В ячейке можно смешивать код Python и команды оболочки (с `!`). Для просмотра документации — `?` после имени функции: `pd.read_csv?`.

Linux

```
# Запуск Jupyter Lab
jupyter lab

# Или классический Jupyter Notebook
jupyter notebook

# Запуск на конкретном порту
jupyter lab --port=8889

# Запуск без открытия браузера (для удалённой работы)
jupyter lab --no-browser --ip=0.0.0.0
```

PowerShell

```
# На Windows запуск аналогичный
jupyter lab

# Запуск с конкретным каталогом
jupyter lab --notebook-dir=C:\Users\USERNAME\projects
```

Jupyter Notebook

```
# Команды оболочки внутри ячейки
!pip list
!python --version
!pwd

# Просмотр документации
pd.read_csv?

# Магические команды Jupyter
%time pd.read_csv("data/sales.csv") # замер времени
%matplotlib inline # графики в ноутбуке
%load_ext autoreload # автоперезагрузка модулей
```

ПРИМЕЧАНИЕ. Магические команды

`%time` замеряет время выполнения, `%%time` — время всей ячейки. `%matplotlib inline` встраивает графики прямо в ноутбук. `%load_ext autoreload` и `%autoreload 2` автоматически перезагружают модули при изменении — удобно при разработке `utils.py`.

8.2. Google Colab

Colab — бесплатный Jupyter в облаке Google. Главное отличие — доступ к Google Drive и GPU/TPU. Файлы сохраняются в Drive или во временной папке `/content/`. Для длительной работы — монтируйте Drive.

Google Colab

```
# Монтирование Google Drive
from google.colab import drive
drive.mount("/content/drive")

# Проверка подключения
import os
print(os.listdir("/content/drive/MyDrive"))

# Установка библиотек (если не входят в стандартный набор Colab)
!pip install pyarrow openpyxl

# Проверка версии Python и окружения
import sys
print("Python:", sys.version)
print("Executable:", sys.executable)
```

Google Colab

```
# Скачивание файла из Colab на локальный компьютер
from google.colab import files
df_clean.to_csv("sales_clean.csv", index=False)
files.download("sales_clean.csv")

# Загрузка файла с локального компьютера в Colab
from google.colab import files
uploaded = files.upload()
for filename in uploaded.keys():
print("Загружен:", filename)
```

ВНИМАНИЕ. Colab перезапускается

Бесплатный Colab перезапускается через 12 часов (или раньше при неактивности). Все переменные теряются. Сохраняйте результаты в Drive, а не в `/content/`. Для длинных вычислений используйте `checkpointing`: периодически сохраняйте промежуточные DataFrame.

8.3. PowerShell (Windows)

PowerShell — стандартная оболочка на Windows. Используется для запуска Python-скриптов, активации виртуального окружения, клонирования репозитория. В PowerShell можно запускать команды git, python, pip без изменений. Виртуальное окружение активируется через `venv\Scripts\Activate.ps1`.

PowerShell

```
# Создание виртуального окружения
python -m venv venv

# Активация
.\venv\Scripts\Activate.ps1

# Установка пакетов
pip install -r requirements.txt

# Запуск Jupyter
jupyter lab

# Клонирование репозитория
git clone https://github.com/USERNAME/REPOSITORY.git
cd REPOSITORY

# Установка зависимостей проекта
pip install -r requirements.txt

# Запуск ноутбука
jupyter notebook notebook/cheatsheet_demo.ipynb

# Запуск Python-скрипта
python scripts/build_pdf.py

# Деактивация виртуального окружения
deactivate
```

ПРИМЕЧАНИЕ. Если PowerShell ругается на Activate.ps1

По умолчанию в PowerShell запрещён запуск скриптов. Разрешите один раз: `Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser`. После этого `Activate.ps1` будет работать.

8.4. Linux и macOS

В Linux и macOS используется bash или zsh. Виртуальное окружение активируется через `source venv/bin/activate`. Команды git, python, pip работают аналогично. На Linux можно использовать системный Python или установить через `pyenv` для нескольких версий.

Linux

```
# Создание виртуального окружения
python3 -m venv venv

# Активация
source venv/bin/activate

# Установка пакетов
pip install -r requirements.txt

# Запуск Jupyter
jupyter lab

# Клонирование репозитория
git clone https://github.com/USERNAME/REPOSITORY.git
cd REPOSITORY

# Установка зависимостей
pip install -r requirements.txt

# Запуск ноутбука
jupyter notebook notebook/cheatsheet_demo.ipynb

# Запуск Python-скрипта
python3 scripts/build_pdf.py

# Деактивация
deactivate
```

СОВЕТ. venv в проекте — лучшая практика

Создавайте `venv` внутри каждого проекта. Тогда зависимости изолированы, и при обновлении одной библиотеки в одном проекте не сломается другой. Файл `requirements.txt` фиксирует точные версии — воспроизводимость гарантирована.

8.5. Клонирование репозитория и первый запуск

Полный сценарий: клонировать репозиторий, создать окружение, установить зависимости, запустить ноутбук. Эти шаги выполняются один раз при первом развертывании проекта на новой машине. Для обновления — `git pull` и при необходимости `pip install -r requirements.txt`.

PowerShell

```
# Полный сценарий на Windows
git clone https://github.com/USERNAME/REPOSITORY.git
cd REPOSITORY
python -m venv venv
.\venv\Scripts\Activate.ps1
pip install -r requirements.txt
jupyter notebook notebook\cheatsheet_demo.ipynb
```

Linux

```
# Полный сценарий на Linux/macOS
git clone https://github.com/USERNAME/REPOSITORY.git
cd REPOSITORY
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
jupyter notebook notebook/cheatsheet_demo.ipynb
```

ВНИМАНИЕ. Не забывайте активировать venv

Если запустить `pip install` без активации `venv`, пакеты установятся в систему. Это приводит к конфликтам и нестабильности. Перед каждой установкой проверяйте: `which python` (Linux) или `where python` (Windows) — путь должен указывать в `venv`.

8.6. Запуск Jupyter в Colab с GitHub

Colab может открыть ноутбук прямо из GitHub. Это удобно для демонстраций и не требует локальной установки. Введите URL GitHub-ноутбука в форму на `https://colab.research.google.com/github/` — Colab откроет его. Если в ноутбуке нужны файлы из репозитория, клонируйте весь репозиторий в `/content/`.

Google Colab

```
# Клонирование репозитория прямо в Colab
!git clone https://github.com/USERNAME/REPOSITORY.git
%cd REPOSITORY

# Установка зависимостей
!pip install -r requirements.txt

# Теперь можно импортировать модули репозитория
import pandas as pd
from utils import quick_inspect # если есть utils.py

# Чтение данных из репозитория
df = pd.read_csv("data/sales.csv")
quick_inspect(df, "df")
```

СОВЕТ. %cd — магическая команда смены каталога

В Colab `%cd` меняет рабочий каталог для всего ноутбука. Обычный `cd` не работает (это команда оболочки, а не Python). После `%cd REPOSITORY` пути вида `data/sales.csv` работают корректно.

9. Итоговый чек-лист

Перед тем как считать работу выполненной, пройдите чек-лист. Он собран из практики: каждая пункта — это типичная ошибка, которую легко пропустить. Отметьте всё, что подтверждено; не отмечайте "на всякий случай" — лучше явно проверить.

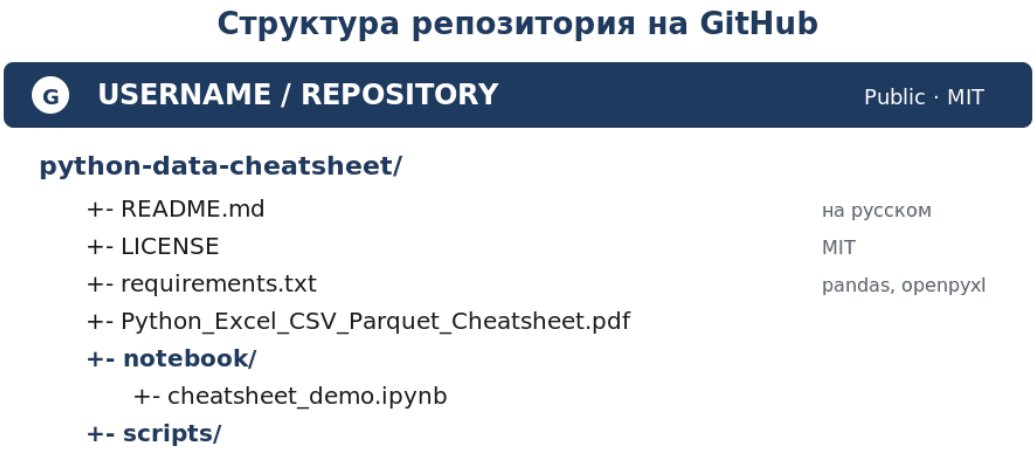
9.1. Чек-лист для PDF-шпаргалки

- Все примеры кода запускаются без ошибок в чистом окружении с `requirements.txt`.
- Имена переменных согласованы: `df_raw`, `df_work`, `df_clean`, `sales_df` используются единообразно во всех разделах.
- Каждый блок кода имеет метку среды выполнения: Jupyter Notebook, Google Colab, PowerShell, Linux или Python.
- Все ссылки на репозиторий используют шаблоны `USERNAME` и `REPOSITORY`, а не реальные имена.
- Версии библиотек актуальны: pandas ≥ 2.0, openpyxl ≥ 3.1, pyarrow ≥ 14.0, matplotlib ≥ 3.7, seaborn ≥ 0.13.
- Разделы идут в логичном порядке: подготовка среды → чтение → осмотр → преобразование → безопасность → визуализация → сохранение → среды.
- Для каждого важного преобразования есть проверка результата (`shape`, `isna().sum()`, `dtypes`).
- В тексте нет таблиц; используется форматирование заголовками, списками и блоками кода.
- Нет несуществующих ссылок на репозитории или файлы.
- Блоки "Совет" и "Внимание" выделены и не перекрывают код.

9.2. Чек-лист для Jupyter Notebook

- Ноутбук запускается сверху вниз без ошибок: `Kernel → Restart & Run All`.
- Все ячейки с кодом имеют комментарии и markdown-пояснения.
- Воспроизводимый набор `sales_df` генерируется в первой ячейке после импортов.
- Демонстрационные данные не требуют внешних файлов — всё генерируется в коде.
- В ноутбуке есть проверки результатов после каждого важного шага.
- Используется дисциплина `df_raw`/`df_work`/`df_clean` во всех примерах.
- В конце ноутбука сохраняются результаты в трёх форматах: `.csv`, `.xlsx`, `.parquet`.
- Импорты согласованы и не дублируются.
- Версии библиотек можно проверить одной ячейкой: `pd.__version__`, `np.__version__` и т.д.

9.3. Чек-лист для репозитория на GitHub



- Файл `README.md` существует, на русском языке, описывает проект и его назначение.
- Файл `requirements.txt` содержит все необходимые библиотеки с минорными версиями.
- Файл `LICENSE` существует и содержит лицензию MIT (или другую выбранную).
- Файл `.gitignore` игнорирует `venv/`, `__pycache__/`, `.ipynb_checkpoints/`, временные файлы.
- PDF-шпаргалка лежит в корне или в `docs/` и доступна для скачивания.
- Исходник PDF (Python-скрипт ReportLab) лежит в `scripts/` и может быть запущен.
- Jupyter Notebook лежит в `notebook/` и запускается.
- В README есть команды для клонирования и запуска на Linux и PowerShell.
- В README есть ссылка на открытие ноутбука в Colab.
- Репозиторий публичный (для общедоступного использования) или имеет описание в README.

9.4. Финальные советы

ПРИМЕЧАНИЕ. Перед публикацией — свежий взгляд

Откройте репозиторий на новой машине, пройдите по шагам README от клонирования до запуска. Если что-то не получается "с нуля" — это сигнал, что в README не хватает важного шага. Хорошо задокументированный проект узнаётся именно по этому тесту.

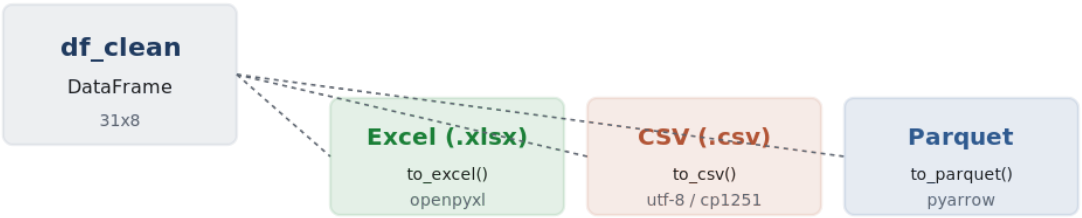
СОВЕТ. Версии в requirements.txt

Используйте точные версии: `pandas==2.2.0`, а не `pandas>=2.0`. Это даёт гарантированную воспроизводимость. Для обновления — отдельный коммит с повышением версий и проверкой.

ВНИМАНИЕ. Не коммитьте большие файлы

PDF размером до 10 МБ — нормально для Git. Больше — лучше через Git LFS или выложить отдельно (например, на GitHub Releases). Файлы данных в `data/` — только если они небольшие и нужны для примеров.

Сохранение DataFrame в три формата



Excel — для отчетов. CSV — для обмена. Parquet — для аналитики